uc3m | Universidad Carlos III de Madrid

University Degree in Data Science and Engineering
2024-2025

Bachelor Thesis

"Design and implementation of a web application for functional spectral data analysis and modeling"

Lucas García García

Tutor:

ALDO LUIS MORENO OYERVIDES

Leganes, September 2025

This work is licensed under Creative Commons **Attribution – Non Commercial – Non Derivatives**

ABSTRACT

This Final Degree Project presents the design and implementation of a local-first, interactive web application for Functional Data Analysis (FDA) of spectral datasets. The platform enables users to upload regularly or irregularly sampled spectra, define the domain, and transform signals into functional objects through basis expansions (B-splines, Fourier, or Wavelets). Once functionalized, users can explore and analyze the data using both unsupervised and supervised approaches, with integrated model diagnostics and interactive visualizations to support model interpretation.

Built with Next.js, Python, and scikit-fda, the tool offers a modern interface based on Plotly for zooming, filtering, and comparing curves, along with exportable results and metadata tracking to ensure reproducibility. Designed for local execution, it guarantees data privacy while providing a modular workflow suitable for research and teaching.

The application was validated with spectral datasets from different domains, demonstrating its usability, accuracy, and potential to streamline FDA workflows without requiring programming skills. The main contribution is a unified, visual pipeline that integrates basis selection, functional modeling, and diagnostics, promoting transparent and reproducible analyses.

Keywords: functional data analysis, spectral data, web application, regression, B-splines, Plotly.

ACKNOWLEDGMENTS

I would like to express my sincere gratitude to Carlos III University of Madrid (UC3M) for providing me with a high-quality academic education and for supplying the necessary resources to carry out this Final Degree Project. I owe special thanks to my supervisor, Aldo Moreno, for his continuous guidance and support throughout the entire project. His expertise and willingness to answer my questions have been essential for the development and successful completion of this work.

I am also grateful to my fellow students at the university for sharing this academic journey with me. Your friendship has made these years truly enjoyable, and your support has been invaluable to overcoming challenges along the way.

Lastly, I want to deeply thank my family for always being there and supporting me unconditionally. Their faith in me and constant encouragement have given me the strength to persevere, even during the most difficult moments. This achievement would not have been possible without their unwavering support.

TABLE OF CONTENTS

1. INTRODUCTION 1
1.1. Motivation 1
1.2. State of the Art 2
1.3. Objectives and Scope 3
2. FDA THEORETICAL BASIS 5
2.1. Functional Representation 5
2.1.1. Basis Expansion 5
2.1.2. Smoothing and Penalization: λ selection 5
2.2. Exploratory Analysis 6
2.2.1. Descriptive Statistics (Mean, Variance, Correlation, Confidence Bands) 6
2.3. Unsupervised techniques 7
2.3.1. Functional Principal Component Analysis 7
2.4. Supervised techniques 8
2.4.1. Functional Principal Component Regression 8
2.4.2. Functional Principal Component Logistic Regression 8
2.4.3. Scalar-on-Function Regression 9
2.4.4. Function-on-Scalar Regression 9
2.4.5. Function-on-Function Regression 9
2.4.6. Functional Linear Discriminant Analysis 10
2.4.7. Supervised Summary 10
2.5. Summary 10
3. DESIGN AND IMPLEMENTATION 12
3.1. Application Workflow and Architecture 12
3.2. Technology and Tool Justification 15
3.2.1. Frontend Framework (Next.js, React, Tailwind, shadcn/ui) 15
3.2.2. Backend (Python for Computational Engine) 16
3.2.3. Middleware/API Layer (Next.js API Routes + Node.js ↔ Python communication) 17
3.2.4. Visualization (Plotly.js) 17
3.2.5. FDA Library (scikit-fda) 18

3.3. Backend Design 19
3.3.1. Data Handling and Validation 19
3.3.2. Functionalization: Basis Expansion and Smoothing 19
3.3.3 Exploratory and Descriptive Analysis 19
3.3.4. Unsupervised Analysis: FPCA 20
3.3.5. Supervised Analysis 20
3.3.6. Results and Export Capabilities 20
3.3.7. Performance and Privacy Considerations 21
3.4. Frontend Design 21
3.4.1. State Management 21
3.4.2. Modular Layouts 21
3.4.3. User Interface Components 22
3.4.4. Interactive Visualization 22
3.4.5. Features Supporting Workflow Iteration 22
3.4.6. Export Capabilities 23
3.5. Reproducibility and Exports 23
3.5.1 Data and Workflow Persistence 23
3.5.2 Export of Results 23
3.5.3 Export of Visualizations 24
3.5.4 Scientific and Practical Justification 24
3.6. Example Workflow 24
3.6.1. Data Import and Domain Definition 25
3.6.2 Functionalization 27
3.6.3 Exploratory Analysis 30
3.6.4 Unsupervised Analysis: FPCA 32
3.6.5 Supervised Analysis 32
4. RESULTS 37
4.1. User Workflow: Data Upload and Domain Configuration 37
4.2. Functional Representation 37
4.3. Exploratory Analysis 37
4.4. Unsupervised Analysis: Functional Principal Component Analysis 38

4.5. Supervised Analysis 38
4.6. Export and Reproducibility 38
4.7. Summary of Functionality 39
5. DISCUSSION AND CONCLUSIONS 40
5.1. General Conclusions 40
5.2. Evaluation of Objectives 40
5.3. Limitations 41
5.4. Future Lines of Work 41
6. REGULATORY FRAMEWORK 42
6.1. Data Privacy and Protection 42
6.2. Ethical and Professional Responsibility 42
6.3. Technical Standards and Good Practices 43
6.4. Intellectual Property and Licensing Strategy 43
6.5. Concluding Remarks on the Regulatory Framework 44
7. SOCIO-ECONOMIC ENVIRONMENT 45
7.1. Impact on Research and Technology Transfer 45
7.2. Contribution to Scientific and Technical Education 45
7.3. Potential Impact on Applied Sectors 45
7.4. Alignment with the Sustainable Development Goals (SDGs) 46
7.5. Estimated Budget and Costs 46
7.6. Final Considerations 47
8. BIBLIOGRAPHY 48
9. ANNEX A – DECLARATION OF GENERATIVE AI USE 51

LIST OF FIGURES

Figure 2.1. Theoretical techniques implemented in the app. ……..…...……11

Figure 3.1. Conceptual workflow of the FDA spectral analysis web application...…....14

Figure 3.2. System architecture of the FDA web platform..…….……..…15

Figure 3.3. Screenshot before data upload……..…….…...25

Figure 3.4. Screenshot after data upload showing summary and features of the dataset……..……26

Figure 3.5. Screenshot of the data upload and domain setup interface with dataset preview. ……...……..…….27

Figure 3.6. Screenshot of the functionalization interface showing smoothed curves with chosen basis and λ……...……28

Figure 3.7. Screenshot of the functionalization interface after saving a dataset and alowing the user to proceed to analysis……...29

Figure 3.8. Screenshot of mean curve with confidence bands and variance plot.....…31

Figure 3.9. Screenshot of FPCA results showing eigenfunctions and variance explained……..…32

Figure 3.10. Screenshot of model summary in FPCLoR...….……..….……..33

Figure 3.11. Screenshot of ROC curve in FPCLoR.…….…….…….34

Figure 3.12. Screenshot of fitted values and residuals in FPCLoR ..…….…….....35

Figure 3.13. Screenshot of functional parameters and confidence intervals in FPCLoR ……..…...36

LIST OF TABLES

Table 1.1. Features comparison between application alternatives and competitors……..3

Table 3.1. Application's architecture layers and main roles……..….15

Table 3.2. GitHub popularity and community size of modern frontend frameworks (2025) …….…….16

Table 3.3. Comparative features of visualization libraries……...…...18

Table 3.4. Comparison of functional data analysis libraries……..…….18

Table 4.1. Summary of download formats……..……39

Table 4.2. Summary of core modules and features implemented……..…….39

Table 5.1. Summary of achieved objectives……...…….40

Table 7.1. Summary of achieved objectives……...…….47

LIST OF ABREVIATIONS

API	Application Programming Interface
AUC	Area Under the Curve
CSV	Comma-Separated Values
FDA	Functional Data Analysis
FPCA	Functional Principal Component Analysis
FPCLoR	Functional Principal Component Logistic Regression
FPCR	Functional Principal Component Regression
GCV	Generalized Cross-Validation
JSON	JavaScript Object Notation
LDA	Linear Discriminant Analysis
MAE	Mean Absolute Error
MIT	Massachusetts Institute of Technology
PCA	Principal Component Analysis
PC	Principal Component
FPC	Functional Principal Component
PLS	Partial Least Squares
RMSE	Root Mean Square Error
ROC	Receiver Operating Characteristic
SD	Standard Deviation
SDGs	Sustainable Development Goals
SME	Small and Medium-sized Enterprise
UC3M	Universidad Carlos III de Madrid

1. INTRODUCTION

1.1. Motivation

In today's scientific and technological landscape, spectroscopy is widely used due to its ability to elucidate the structure of matter and support the development of technologies for remote and non-invasive diagnosis and quantification of molecules. This includes a wide range of applications in biomedicine [1], industrial process monitoring [2], precision agriculture [3], and security [4]. As measurement systems and spectroscopic techniques continue to evolve, the resulting datasets have grown significantly in volume and complexity, often containing overlapping signals from multiple sources or species. Extracting useful information from these datasets is essential to make better scientific and technical decisions.

In this context, chemometric techniques are powerful tools for addressing spectral data. These methods help reduce noise, identify relevant spectral regions, and create predictive models that can automate diagnosis or quantify compounds. When dealing with complex spectral data, researchers may adopt one or more theoretical frameworks, including Multivariate Analysis (MA) [5], Functional Data Analysis (FDA) [6], and Machine Learning (ML) approaches [7]. Each provides a unique family of methods and is more or less suitable depending on the structure of the data and the objectives of the analysis. FDA, in particular, is well suited for spectral data, as it preserves the continuous nature of the measurements by representing them as functions over a domain. This approach helps reduce dimensionality and makes it easier to interpret the results in the original wavelength or time scale, while also allowing direct interpretation of model parameters in their physical domain. For example, studies have shown the successful application of FDA-based models in enhancing signal-to-noise ratio in near-infrared spectroscopy [6], identifying informative wavelength bands for compound prediction [8], and building accurate classification or regression models for disease detection [9].

Despite the theoretical maturity of these frameworks, practical implementations still pose challenges. Popular environments such as MATLAB [10], R [11], and Python [12] offer powerful tools, but these typically require intermediate to advanced programming skills, which can be a barrier for users who are not experienced in coding. Other existing web platforms or graphical tools often have additional limitations, such as the lack of detailed model diagnostics, limited flexibility, or the absence of session saving for continued work.

This Bachelor's Thesis aims to bridge that gap through the design and implementation of an interactive, modular, and scalable web application for Functional Data Analysis. The platform allows users to upload, visualize, explore, and model spectral datasets without writing code or installing additional software. Its intuitive interface includes interactive visualizations with zoom, hover, curve comparisons, and exportable results. Users are guided through a reproducible workflow that includes basis expansions (e.g., cubic B-splines, Fourier, Wavelets), exploratory and unsupervised analysis, and supervised

modeling using regression and classification pipelines. The backend is implemented in Python, using modern FDA libraries (e.g., scikit-fda), and structured as a modular codebase to facilitate future expansion.

This tool is designed to make FDA more accessible, especially for researchers in fields where spectral and functional data are common. It also serves as a flexible base for further research, education, and collaborative exploration in programs such as Marie Skłodowska-Curie training networks. Moreover, this work lays the foundation for a broader project that aims to extend the application's capabilities to a wider range of chemometric techniques, including the previously mentioned frameworks beyond FDA, to cover a more diverse set of data analysis needs.

1.2. State of the Art

The analysis of spectral data has advanced considerably in the last decades thanks to the development of computational tools in both R and Python ecosystems. In R, packages such as "fda", refund, and "fda.usc" provide functional representation, smoothing, and statistical modeling [13][14]. Similarly, Python offers libraries such as "scikit-fda" and "FDApy", which implement similar functionality following a syntax closer to the scikit-learn style [15][16]. These tools allow the creation of flexible and reproducible analysis pipelines when used by experienced programmers.

However, using these libraries effectively requires programming skills, environment setup, and familiarity with version control. For many applied researchers and students, these requirements can be a barrier. As a response, some graphical applications have been developed. One example is STATFDA, a web app that offers an interface to directly load data, apply smoothing techniques, perform Functional Principal Component Analysis (FPCA), and run classification models. While user-friendly, STATFDA has limitations: it is designed with a "static" format in terms of graph display and offers limited control over the adjustment parameters in the analysis [17].

Furthermore, the flexibility to save and compare multiple functionalization strategies, or to integrate outputs with external tools, is rarely present.

In addition to these tools, Table 1.1 also includes the platform developed in this Bachelor's Thesis, presented alongside representative alternatives in the FDA software landscape. The aim of this comparison is to position the proposed solution relative to existing approaches, considering key aspects such as interactivity, diagnostic capabilities, extensibility, privacy, and workflow flexibility.

Table 1.1 presents a comparison of key features between R/Python libraries, interactive web applications, and cloud-first tools. The comparison highlights that, although programming libraries offer the highest flexibility and methodological depth, they require significant coding skills and setup time. Graphical web applications improve accessibility but often sacrifice advanced diagnostics, extensibility, and session persistence. Cloud-first solutions may provide scalability but can conflict with data privacy or governance policies.

Table 1.1. FEATURES COMPARISON BETWEEN APPLICATION ALTERNATIVES AND COMPETITORS

Feature	STATFDA	Scikit-fda	FDApy	Our App
GUI-Based Interface	Yes	No	No	Yes
Interactivity (Zoom, Toggle)	Partial	No	No	Full (Plotly)
Local Execution	No (cloud)	Yes	Yes	Yes (Offline)
No-Code Workflow	Yes	No	No	Yes
Export Options (CSV, PNG, JSON)	Limited	Partial	Partial	Extensive
Save Multiple Datasets	No	No	No	Yes (5 max)
Diagnostics (RMSE, AUC, etc.)	Basic	Partial	No	Full

Source: Own elaboration based on STATFDA.org, package documentation, and platform testing

These findings suggest a clear trend: tools that lower the entry barrier for non-programmers often lack the modularity and diagnostic depth needed for advanced FDA workflows, while those that offer maximum flexibility demand substantial technical expertise. The proposed application seeks to bridge this gap by combining an accessible interface with advanced features usually found only in coding-based environments, delivered in local-first architecture to ensure data privacy.

1.3. Objectives and Scope

The main goal is to design and implement a local-first, interactive web application for FDA of spectral datasets, enabling users to carry out complete analysis workflows without the need for programming skills or complex software installations.

Beyond enabling the FDA pipeline, from preprocessing to exploratory and supervised analysis, the platform is designed to serve as a foundation for future integration of additional analytical families, including MA and ML approaches. To ensure sustainability, the architecture must support local-first execution and provide backend coding that allows for the modular expansion of the system.

The achievement of this general objective can be divided into the following specific objectives:

- Design modular and scalable architecture. Design the backend architecture to facilitate integration of additional FDA methods and, in future developments, frameworks beyond FDA, including MA and ML approaches.
- Integrate robust and well-documented FDA libraries. Include “scikit-fda” to implement basis expansion techniques (cubic B-splines, Fourier, and Wavelets),

smoothing, and dimensionality reduction. Provide adapters to accommodate future extensions with multivariate or machine learning methods.

- Develop an interactive and user-friendly interface. Use of modern frontend frameworks (Next.js, Tailwind, Plotly) that provide dynamic plots with zoom, filtering, component toggling, and custom tooltips, allowing direct exploration of functional data and model outputs.
- Support comparative analysis. Allow users to generate, save, and switch between multiple functionalization using different basis types and smoothing parameters. All user configurations should be recorded and made exportable for auditing and reproducibility.
- Model diagnostics and export results. include interactive error metrics, like Root Mean Squared Error (RMSE), coefficient of determination (R^2), Mean Absolute Error (MAE), Area Under the Curve (AUC), and options to export functional data, model scores, plots, and coefficients in multiple formats (CSV, PNG).
- Implement workflow validation mechanisms so that each analytical tool (e.g., FPCA, regression, classification) becomes available only when the required inputs are present. Warnings and guidance messages should prevent improper use or misinterpretation of results.
- Provide lightweight decision-support messages based on quantitative metrics such as Generalized Cross-Validation (GCV), explained variance ratios, and residual distributions. These messages are designed to assist, but not replace, expert interpretation of the data.

2. FDA THEORETICAL BASIS

FDA is a statistical framework for analyzing data that vary continuously over a domain, such as time, space, or wavelength. Unlike traditional multivariate analysis, which treats observations as finite-dimensional vectors, FDA considers them as realizations of smooth functions. This approach allows for a more natural representation of many real-world phenomena and enables mathematical operations such as differentiation, integration, and the application of operators over function spaces.

The mathematical foundation of FDA is based on Hilbert space theory, particularly the space of square-integrable functions $L^2(\mathcal{T})$, which forms a complete inner product space. In this setting, each observation (measured spectrum) is represented as a function $x_i(t) \in L^2(\mathcal{T})$, where $\mathcal{T} \subset \mathbb{R}$ is the domain of interest. This structure makes it possible to apply orthogonal decompositions and regularization techniques to build interpretable models for complex datasets [6][8].

In the remainder of this chapter, we describe the main FDA tools integrated into the developed web application. These include functional representations, exploratory analysis techniques, unsupervised dimensionality reduction, and supervised regression and classification models. Each technique is introduced at a conceptual level, providing the theoretical basis for the implementation described in later chapters.

2.1. Functional Representation

2.1.1. Basis Expansion

Functional data often originates from discrete measurements and must be transformed into smooth functions before analysis. A common approach is the use of basis expansion techniques, which approximate each observation $x_i(t)$ as:

$$x_i(t) \approx \sum_{k=1}^{K} c_{ik}\phi_k(t). \tag{2.1}$$

Here, $\{\phi_k(t)\}_{k=1}^{K}$ are predefined basis functions (e.g., B-splines, Fourier, or Wavelets), and c_{ik} are the coefficients associated with each basis function. The choice of basis depends on the nature of the data:

- **B-splines** are well suited for local adaptation and handling irregular patterns.
- **Fourier** bases are ideal for periodic signals.
- **Wavelets** are useful for capturing local anomalies and multi-resolution structures.

2.1.2. Smoothing and Penalization: λ selection

When fitting functions from noisy or high-dimensional spectral data, it is important to control the trade-off between fidelity to the observed data and smoothness of the estimated functions. This is typically achieved using penalized least squares, minimizing:

$$\mathrm{PEN} = \sum_{i=1}^{n} |x_i(t) - \hat{x}_i(t)|^2 + \lambda \int_T \left(\frac{d^2 x_i(t)}{dt^2}\right)^2 dt. \tag{2.2}$$

The first term measures the fit to the observed data, while the second penalizes excessive curvature of the estimated functions. The smoothing parameter λ controls this trade-off:

- Small values of λ lead to functions that follow the data closely, potentially overfitting.
- Large values of λ enforce smoother curves, possibly over smoothing the data.

An objective method to choose λ is GCV, which balances model fit and smoothness by approximating leave-one-out cross-validation in a computationally efficient way. In the developed application, this functionality is available through an Auto (GCV) option, which automatically estimates the optimal smoothing parameter for the uploaded dataset, ensuring stable and reproducible functional representations.[6][13].

2.2. Exploratory Analysis

Exploratory analysis techniques in FDA provide tools for visualizing, summarizing, and understanding the main features of functional datasets before applying more complex statistical models. These methods allow the analyst to describe the average behaviour of functions, quantify variability, and assess local or global patterns across the functional domain.

In the developed web application, exploratory tools are integrated as the first step of analysis after functionalization, offering users an intuitive way to gain insights into their spectral or temporal data.

2.2.1. Descriptive Statistics (Mean, Variance, Correlation, Confidence Bands)

A first step in functional data exploration is the estimation of summary statistics across the observed functions:

- Mean Function: the average behaviour of the functional sample is captured by the pointwise mean:

$$\bar{\boldsymbol{x}}(\boldsymbol{t}) = \frac{\boldsymbol{1}}{\boldsymbol{n}} \sum_{\boldsymbol{i}=\boldsymbol{1}}^{\boldsymbol{n}} \boldsymbol{x}_{\boldsymbol{i}}(\boldsymbol{t}) \tag{2.3}$$

  which provides a central tendency curve.

- Pointwise Variance: local variability is quantified by the variance function:

$$\mathrm{Var}(\boldsymbol{t}) = \frac{\boldsymbol{1}}{\boldsymbol{n}-\boldsymbol{1}} \sum_{\boldsymbol{i}=\boldsymbol{1}}^{\boldsymbol{n}} \left(\boldsymbol{x}_{\boldsymbol{i}}(\boldsymbol{t}) - \bar{\boldsymbol{x}}(\boldsymbol{t})\right)^{\boldsymbol{2}} \tag{2.4}$$

  highlighting regions of the domain where curves exhibit greater dispersion.

- Confidence Bands: **c**onfidence intervals around the mean function can be constructed using pointwise variance, offering a graphical assessment of the uncertainty in the average curve.

- Correlation Functions: pairwise correlations between curves, or within specific regions of the domain, help assess whether subsets of observations exhibit similar behaviours.

These descriptive tools provide both global insights (mean and variance across the entire domain) and local insights (confidence bands and correlation at specific points). In spectral applications, for example, they reveal wavelength intervals with high variability or significant differences in shape.

The integration of these methods into the application allows users to visualize trends, detect anomalies, and identify domain regions of interest before proceeding with unsupervised or supervised analysis [6][19].

2.3. Unsupervised techniques

Unsupervised analysis in FDA is mainly aimed at dimensionality reduction and pattern discovery without relying on external variables. The key technique implemented in the developed application is FPCA, which extends classical PCA to functional data.

FPCA provides a compact representation of a set of functions, enabling the identification of the main modes of variation across the sample. This makes it possible to replace high-dimensional functional data with a reduced number of scores, which summarize the contribution of each function to the principal directions of variability.

2.3.1. Functional Principal Component Analysis

Given a sample of functions $\{x_i(t)\}_{i=1}^{n}$, FPCA expresses each observation as a decomposition of the mean function and a weighted sum of FPCs:

$$x_i(t) = \mu(t) + \sum_{k=1}^{K} \xi_{ik}\ \phi_k(t) \tag{2.5}$$

Where $\mu(t)$ is the mean function, $\phi_k(t)$ are the eigen functions (functional principal components), and $\boldsymbol{\xi_{ik}}$ are the scores for each sample (observation), indicating the contribution of each principal component.

The eigenfunctions are solutions of the integral equation associated with the covariance operator:

$$\int_T C(s,t)\ \phi_k(s)\, ds = \lambda_k\, \phi_k(t) \tag{2.6}$$

Where $C(s,t)$ is the covariance function and λ_k are the corresponding eigenvalues. The eigenvalues represent the variance explained by each component. FPCA thus allows us to:

- Reduce dimensionality: by retaining only the first K components, which typically capture most of the variability in the data.
- Visualize variability: the eigenfunctions highlight the dominant modes of variation across the domain (e.g., regions of the spectrum where curves differ most).

- Compute scores: which summarize each function as coordinates in the reduced principal component space.

In the application, FPCA is implemented with options to:

- Select the number of retained components K.
- Apply varimax rotation, a common post-processing step in component analysis. It is an orthogonal rotation method that maximizes the variance of the squared loadings within each component. In practice, this leads to components where each variable (or region of the functional domain) loads highly on a small number of components and weakly on others. This improves interpretability, since it produces components that are easier to associate with specific localized features of the spectrum, without altering the total variance explained.
- Scale scores when necessary for regression or classification models.

Moreover, FPCA is not only an exploratory tool but also forms the basis for supervised models, such as Functional Principal Component Regression (FPCR) and Functional Principal Component Logistic Regression (FPCLoR), described in Section 2.4 [6][8][18].

2.4. Supervised techniques

Supervised analysis in FDA combines functional predictors with response variables (scalar, binary, categorical, or functional) to build predictive models. Unlike exploratory or unsupervised techniques, supervised methods explicitly aim to explain or predict the behaviour of a response variable from functional data.

In the developed application, several supervised approaches have been implemented, covering regression and classification contexts. Each method adapts classical statistical techniques to the functional setting, often relying on basis expansion or FPCA scores to reduce dimensionality and ensure model stability.

2.4.1. Functional Principal Component Regression

FPCR is a regression technique in which the functional predictor $x_i(t)$ is first reduced via FPCA, and the resulting scores ξ_{ik} are then used as predictor in a linear regression model:

$$y_i = \beta_0 + \sum_{k=1}^{K} \beta_k \, \xi_{ik} + \varepsilon_i \tag{2.7}$$

This method is effective in reducing multicollinearity and preserving interpretability when modeling a scalar response variable. It has been successfully applied in [18] to model spectral responses in biomedical studies, showing its suitability for data with highly correlated functional predictors.

2.4.2. Functional Principal Component Logistic Regression

When the response variable is categorical (e.g., class label), logistic regression can be applied to the FPCA scores:

$$logit\big(P(y_i = 1)\big) = \beta_0 + \sum_{k=1}^{K} \beta_k \, \xi_{ik} \tag{2.8}$$

Here, ξ_{ik} are the scores from the K selected functional principal components, and β_k are the corresponding regression coefficients. FPCLoR provides good classification performance while maintaining the interpretability of domain-specific functional features, making it suitable for applications where both predictive accuracy and feature interpretability are important [6].

2.4.3. Scalar-on-Function Regression

In Scalar-on-Function Regression (SoFR), the response variable is scalar ($y_i \in R$) while the predictor is functional ($x_i(t)$). The model is expressed as:

$$y_i = \alpha + \int_T \beta(t) \; x_i(t) \, dt + \varepsilon_i \tag{2.9}$$

Where $\beta(t)$ is the functional coefficient function, describing the influence of different regions of the domain on the scalar response; and ε_i is the error term.

SoFR is particularly useful when modeling continuous outcomes (e.g., predicting temperature from spectral curves). In practice, $\beta(t)$ is expanded in a basis system, enabling estimation via least squares or penalized regression.

2.4.4. Function-on-Scalar Regression

Function-on-Scalar Regression (FoSR) considers the inverse setting: the response is a function ($y_i(t)$), and the predictors are scalar covariates ($z_i \in R^p$). The model can be written as:

$$\boldsymbol{y_i(t) = \alpha(t) + \sum_{j=1}^{p} z_{ij} \; \beta_j(t) + \varepsilon_i(t)} \tag{2.10}$$

Where $\boldsymbol{\beta_j(t)}$ are functional regression coefficients describing how each scalar predictor influences the response over the domain.

This approach is common in longitudinal or spectral applications where, for instance, treatment groups or experimental conditions are scalar covariates, and the interest lies in their effect across the functional domain.

2.4.5. Function-on-Function Regression

Function-on-Function Regression (FoFR) generalizes regression to settings where both the predictor and the response are functional:

$$\boldsymbol{y_i(t) = \alpha(t) + \int_T x_i(s) \; \beta(s,t) \, ds + \varepsilon_i(t)} \tag{2.11}$$

Where $\beta(s,t)$ is a bivariate coefficient surface that captures how the predictor function at domain point s influences the response function at point t.

FoFR is computationally demanding, but powerful when both input and output vary continuously. In practice, dimensionality reduction (e.g., via FPCA) is applied to both predictor and response functions before estimation.

2.4.6. Functional Linear Discriminant Analysis

FLDA extends classical Linear Discriminant Analysis (LDA) to functional data. Given two or more classes, the goal is to find a discriminant function that maximizes separation between groups while minimizing within-group variability.

In the functional case, each curve is projected onto a discriminant direction $\delta(t)$:

$$\boldsymbol{\eta_i = \int_T x_i(t)\ \delta(t)\, dt \eta_i = \int_T x_i(t)\ \delta(t)\, dt} \tag{2.12}$$

and classification is performed using the projected scores η_i.

FLDA is well suited for binary or multiclass problems with functional predictors, providing interpretable discriminant functions that highlight the most discriminative regions of the domain.

2.4.7. Supervised Summary

The supervised techniques integrated into the application cover a wide spectrum of scenarios:

- SoFR, FPCR: scalar responses (continuous outcomes).
- FPCLoR, FLDA: categorical or binary responses (classification).
- FoSR, FoFR: functional responses (time-varying or spectral outcomes).

Together, these methods enable flexible modelling of diverse real-world problems where predictors and/or responses are functions. By leveraging basis expansions and FPCA scores, the models achieve dimensionality reduction, interpretability, and statistical rigor.

2.5. Summary

FDA provides a complete statistical framework for handling datasets where observations are functions rather than finite-dimensional vectors. The theoretical basis lies in Hilbert space representations, allowing functional objects to be approximated through basis expansions and regularized via smoothing techniques such as penalized least squares or automatic criteria like GCV. This representation is the foundation for all subsequent analyses.

Exploratory analysis tools, including mean functions, variance functions, correlations, and confidence bands, enable the detection of global patterns, group differences, and variability across the domain. These methods offer an intuitive first step before applying more advanced models.

For dimensionality reduction, FPCA plays a central role. FPCA decomposes functional variability into orthogonal components, facilitating visualization, noise reduction, and serving as a basis for supervised models.

Supervised analysis extends FDA to predictive contexts:

- SoFR and FPCR address scalar responses, using functional predictors directly or via FPCA scores.
- FoSR and FoFR capture dynamic relationships when the response itself is functional.
- FPCLoR and FLDA adapt FDA for classification tasks with binary or categorical responses.

Together, these methods form a coherent toolkit for analyzing spectral data. The developed application integrates all these stages, from functional representation to exploration, unsupervised learning, and supervised modelling, in a modular and interactive environment. The inclusion of downloadable outputs (coefficients, discretized functions, FPCA loadings, fitted values, and functional parameters) ensures reproducibility and transparency.

This integration of descriptive, unsupervised, and supervised techniques makes the application a versatile platform for both research and applied domains. Moreover, its modular design allows future extensions to incorporate additional FDA methodologies as required by specific datasets or scientific objectives [6][8][18][19].

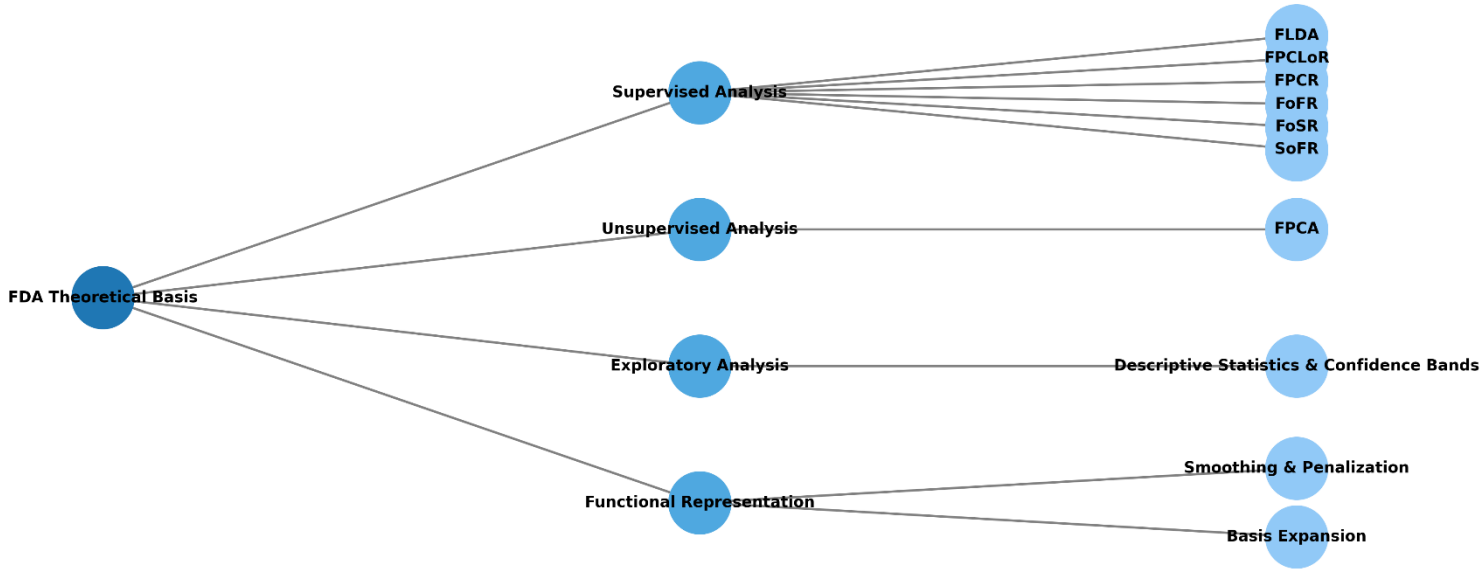

Figure. 2.1. THEORETICAL TECHNIQUES IMPLEMENTED IN THE APP.

3. DESIGN AND IMPLEMENTATION

This chapter presents the design and implementation of the developed web application for spectral data analysis using FDA. The system was conceived to combine mathematical rigor with an intuitive user interface, ensuring that both domain experts and non-specialists can explore, model, and interpret complex functional datasets.

The chapter is structured to first describe the overall workflow and architecture of the platform, which integrates data upload, preprocessing, functionalization, exploratory analysis, and predictive modelling. We then provide a justification for the chosen technologies, followed by detailed descriptions of the backend and frontend design. Special emphasis is placed on reproducibility and result exports, as these features are crucial for academic and applied research. Finally, an example workflow illustrates how a user interacts with the system to perform a complete FDA pipeline.

3.1. Application Workflow and Architecture

The application follows a modular workflow that mirrors the logical stages of Functional Data Analysis. Each step is implemented as an independent module, allowing users to move back and forth without losing progress. This design ensures both flexibility and traceability, as adjustments made in earlier steps automatically propagate to subsequent analyses. At each step, no wrong actions can be made, that is, the user can only proceed to the following step if everything is done as expected. If this is not the case, a warning appears, explaining why the user cannot proceed, and what he should do to fix it.

The workflow can be summarized in eight main stages (Figure. 3.1):

1. Data Upload: The user imports spectral datasets (and optional scalar/response variables) in CSV or TXT format. The system validates delimiters, headers, and sampling structure.
2. Validation: Automatic summaries and warnings ensure that the data structure is suitable for functionalization.
3. Domain Setup: The domain of interest (e.g., wavelength range) is defined by start, end, and resolution.
4. Functionalization: Raw discrete data are transformed into smooth functional objects using basis expansion (B-splines, Fourier, Wavelets) with smoothing parameter λ, which can also be optimized via Generalized Cross Validation (GCV).
5. Exploratory Analysis: The system provides descriptive statistics such as mean functions, variance curves, and confidence bands to help understand group structures.
6. Unsupervised Analysis: Functional Principal Component Analysis (FPCA) reduces dimensionality and identifies main modes of variability.

7. Supervised Analysis: Predictive models are fitted depending on the response type. Implemented methods include Scalar-on-Function Regression, Function-on-Scalar Regression, Function-on-Function Regression, Functional Principal Component Regression, Functional Principal Component Logistic Regression, and Functional Linear Discriminant Analysis.
8. Results and Exports: Outputs such as fitted values, residuals, and functional parameters $\beta(t)$ can be visualized and downloaded in multiple formats (CSV, PNG, SVG).

This conceptual workflow is depicted in Figure. 3.1, while the technical three-layer architecture of the system (Frontend, Middleware, Backend) is shown in Figure. 3.2. Together, these diagrams highlight both the logical flow for the user and the software architecture that supports it.

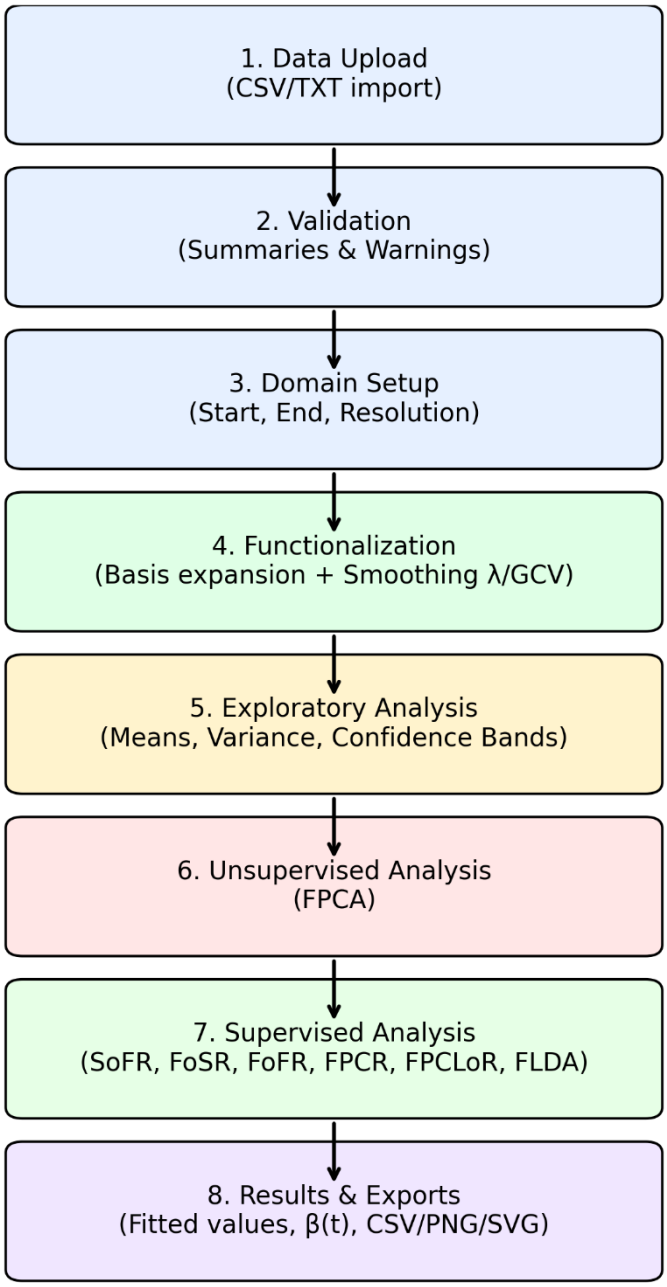

Figure. 3.1. CONCEPTUAL WORKFLOW OF THE FDA SPECTRAL ANALYSIS WEB APPLICATION.

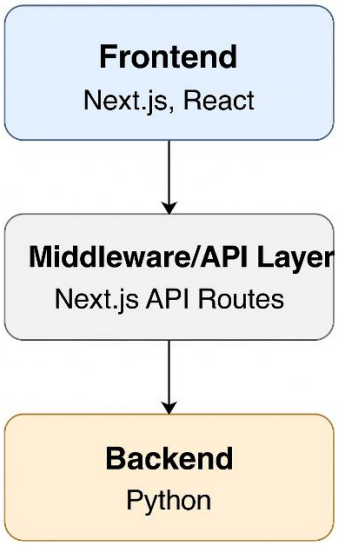

Figure. 3.2. SYSTEM ARCHITECTURE OF THE FDA WEB PLATFORM.

Table 3.1 summarizes the technologies used and their specific role in the system. Unlike traditional client–server architectures, the backend in this application is hybrid: validation and orchestration tasks are handled by the Next.js API layer (Node.js), while Python serves as a dedicated computational engine for all functional data analysis routines. This will be explained in more detail in the following section.

Table 3.1. APPLICATION'S ARCHITECTURE LAYERS AND MAIN ROLES

Layer	Technology/Tool	Main Role
Frontend	Next.js, React, TypeScript, TailwindCSS, shadcn/ui, Plotly.js	UI, navigation, data input, visualization
Middleware	Next.js API Routes (Node.js)	Data validation, orchestration, file handling, communication with Python
Backend	Python 3.11+, scikit-fda, numpy, pandas, ml-pca	Functional representation, smoothing, FPCA, supervised/unsupervised models

3.2. Technology and Tool Justification

To ensure both scientific validity and usability, the platform relies on a carefully selected technology stack that balances performance, extensibility, and user experience. The choice of tools reflects the need for advanced numerical computation, interactive web-based visualization, and modular design. Below, we justify the main technologies employed in the development of the application.

3.2.1. Frontend Framework (Next.js, React, Tailwind, shadcn/ui)

The front end of the application was implemented using Next.js 14, a React-based framework widely recognized for its performance, scalability, and hybrid rendering

capabilities (static generation + server-side rendering). This allows seamless integration of both static and dynamic content, ensuring fast responses even when handling large datasets.

User interface design was built with Tailwind CSS and shadcn/ui, which provide a highly customizable, component-driven styling system with accessibility standards built in. This combination enabled the rapid development of reusable interface elements, such as parameter sliders, tabbed dashboards, and responsive layouts.

Compared with alternatives such as Angular or Vue.js, Next.js was selected due to its strong community support, maturity, and native TypeScript integration, which improves maintainability and reduces runtime errors. These advantages are reflected in industry benchmarks where Next.js consistently ranks among the top frameworks for production-grade applications [20][21][22]. Table 3.2 presents a comparative overview of the popularity of modern frontend frameworks.

Table 3.2. GITHUB POPULARITY AND COMMUNITY SIZE OF MODERN FRONTEND FRAMEWORKS (2025)

Framework	GitHub Stars (Thousands)	Description
React	236	Most popular, backed by Meta (Facebook), huge ecosystem
Vue.js	209	Flexible, easy to learn, strong community
Angular	98	Enterprise focused, full-featured, backed by Google
Svelte	83	Rising star for speed and simplicity
Solid.js	30	Fine-grained reactivity, growing popularity
Preact	35	Lightweight React alternative

Source: Adapted from [20][21][22]

3.2.2. Backend (Python for Computational Engine)

The computational backend was developed in Python 3.11, which executes all Functional Data Analysis (FDA) methods, including:

- Basis expansion (B-splines, Fourier, Wavelets).
- Penalized smoothing with both manual λ selection and automatic λ selection via GCV.
- Exploratory analysis (mean functions, variance, confidence bands).
- Unsupervised analysis (Functional PCA).

- Supervised models (Scalar-on-Function Regression, Function-on-Scalar Regression, Function-on-Function Regression, FPCR, FPCLoR, and FLDA).

Python was selected over R because of its versatility, seamless integration with modern web frameworks, and rich scientific ecosystem. Core dependencies include scikit-fda for FDA-specific functionality, NumPy and Pandas for data manipulation, and ml-pca for efficient dimensionality reduction.

While R remains a strong option in academic FDA research, the growing dominance of Python in data science and machine learning communities made it the most appropriate choice for building a modern, extensible, and web-compatible platform [23].

3.2.3. Middleware/API Layer (Next.js API Routes + Node.js ↔ Python communication)

The platform adopts a hybrid backend design. The Middleware/API layer, implemented via Next.js API Routes (Node.js), acts as the orchestrator between the frontend and Python computational engine. Its responsibilities include:

- Validation of uploaded data structures (dimensions, missing values, irregular sampling).
- Management of application state and workflow control.
- Secure communication with Python processes via child process calls in Node.js.

This design avoids the need for a dedicated Python server (e.g., Flask, Django) while maintaining full flexibility and modularity. All heavy computations are delegated to Python scripts, while the middleware ensures stability and isolates the user-facing interface from potential runtime errors.

This hybrid setup was chosen because it maximizes performance and keeps the system lightweight for local execution, while still allowing effortless migration to a cloud or collaborative server environment if required in future versions.

3.2.4. Visualization (Plotly.js)

Interactive visualization is critical in FDA applications. The platform integrates Plotly.js, a modern JavaScript library that supports highly interactive plots with features such as hover tooltips, zooming, panning, and exporting figures in multiple formats (PNG, SVG, HTML).

Compared to Matplotlib or Seaborn, which are primarily static, Plotly enables direct user interaction with spectral functions, functional parameters $\beta(t)$, and diagnostic plots. This improves interpretability and enhances the exploratory capabilities of the platform [23][24]. Table 3.3 compares key features of visualization libraries.

Table 3.3. COMPARATIVE FEATURES OF VISUALIZATION LIBRARIES

Feature	Plotly	Matplotlib	Seaborn
Interactivity	✓	✗	✗
Hover Tooltips	✓	✗	✗
Zoom/Brush	✓	✗	✗
Export (PNG/SVG)	✓	✓	✓
Legend Toggle	✓	✗	✗

Source: Adapted from [24]

3.2.5. FDA Library (scikit-fda)

Among existing FDA libraries in Python, **scikit-fda** was selected due to its rich documentation, sklearn-style API, and active community. Unlike alternatives such as FDApy (Python) or fda (R), scikit-fda provides:

- A wide range of basis expansion methods.
- Integrated smoothing techniques.
- Compatibility with scikit-learn pipelines, making it extensible to machine learning workflows.
- Strong maintenance and community support.

This ensured scientific reliability while facilitating integration into a production-grade web application. Table 3.4 presents a comparative summary of functional data analysis libraries.

Table 3.4. COMPARISON OF FUNCTIONAL DATA ANALYSIS LIBRARIES

Feature	scikit-fda	FDApy	R fda
Basis Expansions	✓	✓	✓
Smoothing (Penalized LS)	✓	✓	✓
sklearn-style API	✓	✗	✗

Web Compatibility	✓	✗	✗
Community & Docs	★★★★	★★	★★★★

Source: Ferraty and Vieu (2022) and official documentation [25][26]

3.3. Backend Design

The backend constitutes the computational core of the application. It was implemented in Python 3.11, serving as the execution engine for all FDA operations. Its modular structure ensures flexibility, maintainability, and scalability, while enabling seamless integration with the Next.js middleware layer.

The backend is organized into several well-defined modules, each corresponding to one stage of the analysis workflow. This modularity makes it possible to extend the platform with new FDA techniques or preprocessing routines without altering the existing logic.

3.3.1. Data Handling and Validation

The backend receives **pre-processed** data from the middleware after an initial structural validation step. Once data files (CSV or TXT) are loaded, Python routines check for:

- Dimensional consistency (number of samples × number of points).
- Missing or non-numeric values.
- Regular vs. irregular sampling.

Validation routines return clear error messages to the middleware, which ensures that the user is guided to correct input issues before proceeding to functionalization.

3.3.2. Functionalization: Basis Expansion and Smoothing

The first computational step in the backend is the functionalization of raw spectral data. This is achieved via basis expansion, approximating each curve as a linear combination of basis functions (B-splines, Fourier, or Wavelets). The user can configure the number of basis functions depending on the smoothness and variability of the dataset.

To prevent overfitting, penalized least squares is applied, with the smoothing parameter λ controlling the balance between fidelity to the data and smoothness of the resulting function. The backend also supports automatic λ selection using GCV, which evaluates candidate λ values and selects the one that minimizes prediction error.

3.3.3 Exploratory and Descriptive Analysis

The backend provides descriptive statistics and exploratory FDA tools, including:

- Mean and variance functions.
- Pointwise confidence bands.
- Correlation analysis between curves.

These outputs allow the user to visually and quantitatively explore the main characteristics of the functional dataset before applying advanced models.

3.3.4. Unsupervised Analysis: FPCA

For dimensionality reduction, the backend implements FPCA. Using the covariance decomposition of functional data, FPCA extracts orthogonal eigenfunctions and sample scores. These components capture the dominant modes of variation across the dataset.

The Python implementation leverages scikit-fda and ml-pca, ensuring numerical stability and integration with other machine learning pipelines. FPCA serves as a foundation for subsequent supervised models (e.g., FPCR, FPCLoR).

3.3.5. Supervised Analysis

The backend integrates a wide range of supervised FDA models, each tailored to different types of response variables:

- SoFR: models' scalar outcomes (e.g., biochemical concentration) from functional predictors.
- FoSR: models' functional responses from scalar predictors.
- FoFR: both predictors and responses are functional.
- FPCR: regression on FPCA scores to handle multicollinearity.
- FPCLoR: classification with categorical responses.
- FLDA: supervised classification using FDA extensions of discriminant analysis.

Each model is implemented with options to configure the number of components and regularization parameters. Results include coefficients, fitted values, residuals, diagnostic plots, and functional parameter estimates $\beta(t)$ with confidence intervals.

3.3.6. Results and Export Capabilities

Once analyses are completed, results are returned to the middleware in structured JSON format. This includes:

- Numerical results: coefficients, scores, fitted values, residuals, and metrics (R^2, RMSE, accuracy, AUC, etc.).
- Functional parameters: $\beta(t)$ curves with confidence intervals.
- Diagnostics: residual analysis and exploratory plots.

The backend also prepares exportable datasets that can be downloaded by the user in CSV or JSON format. These include:

Fitted values and residuals for supervised models.

Basis coefficients and discretized functions for exploratory analysis.

FPCA loadings, scores, and variance tables.

3.3.7. Performance and Privacy Considerations

A key design decision was to ensure that all backend computations are executed locally on the user's device. This guarantees:

- Full data privacy, as no information is uploaded to external servers.
- Low latency, since there are no network round trips.
- Reproducibility, as the same Python routines can be run deterministically on any compatible system.

At the same time, the modularity of the backend allows effortless migration to a server-based or cloud environment if future collaborative use cases demand it.

3.4. Frontend Design

The frontend of the application was implemented using Next.js 14 and React, providing an interactive and modular environment for functional data analysis. The design philosophy was **centred** on usability, modularity, and responsiveness, ensuring that users can perform complex statistical analyses through a clean and intuitive graphical interface.

3.4.1. State Management

A global state is maintained using Zustand and a custom useAppState hook. This architecture centralizes application state**,** such as uploaded datasets, domain configuration, selected functional bases, smoothing parameters, and analysis results**,** and makes it accessible across all interface components.

- This avoids redundancy, prevents data loss during navigation, and enables the user to revisit any step of the workflow without resetting progress.
- For example, after running a supervised analysis, users can return to the Functionalize stage, create a new dataset, and re-run analyses without losing previously generated results.

3.4.2. Modular Layouts

The frontend is structured into dedicated layouts corresponding to each major workflow stage:

- Functionalize View: Interface for basis expansion, smoothing, and functional representation of raw spectral data. The panel includes interactive controls for basis selection, smoothing parameter λ, and automatic λ selection via GCV.
- Analysis Dashboard: Central hub for all exploratory, unsupervised, and supervised FDA techniques. Tabs provide access to descriptive statistics, FPCA, and supervised regression/classification models. Each technique is only activated when its input requirements are satisfied.
- Navigation Controls: Dedicated buttons allow users to move forward in the workflow or return to previous steps (e.g., returning to Functionalize without losing progress). This design mimics the iterative process of real-world analysis.

3.4.3. User Interface Components

The visual interface relies on Tailwind CSS and shadcn/ui, ensuring a modern design, accessibility, and responsiveness across devices. Key features include:

- Interactive forms for uploading data and configuring analysis parameters.
- Dynamic tables for presenting coefficients, residuals, and metrics.
- Tabbed navigation for organizing outputs of each method (e.g., model summary, fitted functions, residual analysis, functional parameters).
- Badges and alerts to highlight results, errors, or important model statistics.

3.4.4. Interactive Visualization

The frontend integrates Plotly.js for all graphical components, enabling:

- Hover tooltips and legends to identify curves or components.
- Zooming, panning, and toggling of series visibility.
- Exporting plots in publication-ready formats (PNG, SVG, HTML).
- Multiple coordinated views, such as:
    - Residuals vs. fitted values and residual histograms.
    - Fitted vs. observed functions.
    - Functional parameter $\beta(t)$ with 95% confidence intervals.
    - FPCA loadings, scores, and variance explained.

These features provide an exploratory environment enhanced with local-first performance and modern web usability.

3.4.5. Features Supporting Workflow Iteration

Special care was taken to replicate and improve upon the iterative workflow observed in traditional FDA environments:

- Auto λ (GCV) button: Automatically selects the optimal smoothing parameter, sparing the user from manual trial-and-error.
- Back to Functionalize button: Allows navigation back to preprocessing without discarding previous results.
- Conditional activation of analysis modules: Ensures users only run techniques when the required input data are valid, preventing runtime errors and misinterpretation.

3.4.6. Export Capabilities

In parallel to backend export functions, the frontend provides download buttons in every analysis module:

- Supervised models: export of fitted values and functional parameters β(t).
- Descriptive analysis: export of basis coefficients and discretized functions.
- FPCA: export of FPC loadings, basis coefficients of loadings, and FPC scores.

These features guarantee reproducibility and facilitate integration of results into external reports or academic publications.

3.5. Reproducibility and Exports

One of the fundamental objectives of the developed platform is to guarantee reproducibility of all functional data analyses. In line with good scientific practices, the application was designed to ensure that every stage of the workflow**,** from data import to model fitting and visualization**,** can be documented, replicated, and exported in standard formats.

3.5.1 Data and Workflow Persistence

- Uploaded datasets, domain definitions, and functional representations are stored in the application state and preserved during the entire workflow.
- Users can return to previous stages (e.g., Functionalize) without losing their progress or results already obtained in subsequent stages.
- The system avoids hidden preprocessing: every parameter selected by the user (basis type, number of components, λ value, etc.) is explicitly visible and recorded.

3.5.2 Export of Results

To support transparency and integration with external environments, the platform allows exporting both numerical and graphical results:

- Supervised Models (SoFR, FoSR, FoFR, FPCR, FPCLoR, FLDA):

    - Fitted values (predictions for each observation).
    - Functional parameters $\beta(t)$ with their confidence intervals.
- Exploratory Analysis (Descriptive statistics):
    - Basis coefficients of the functional representation.
    - Discretized functions, preserving their evaluated points across the domain.
- Unsupervised Analysis (FPCA):
    - Discretized FPC loadings.
    - Basis coefficients of FPC loadings.
    - FPC scores for each observation.

All exports are provided as CSV or JSON files, ensuring compatibility with common statistical software (R, Python, MATLAB) and with spreadsheet applications.

3.5.3 Export of Visualizations

Visualizations are generated with Plotly.js and can be exported directly from the interface in multiple formats:

PNG and SVG for inclusion in publications or reports.

HTML with full interactivity, enabling sharing of dynamic plots with collaborators without requiring them to install the platform.

3.5.4 Scientific and Practical Justification

These export features with a local-first architecture guarantee:

Data privacy: no external servers are involved.

Reproducibility: any analysis can be exactly replicated by reusing the exported datasets and parameters.

Flexibility: exported results can be incorporated into academic theses, publications, or industrial reports without additional processing.

By combining explicit parameter control, workflow persistence, and comprehensive export options, the application ensures that results are not only interpretable but also reusable in future studies and collaborative environments.

3.6. Example Workflow

To illustrate the complete functionality of the developed platform, we describe a typical user scenario in which spectral data are processed, functionalized, and analysed using both exploratory and supervised techniques. This example demonstrates how the different modules interact in a coherent workflow, emphasizing the reproducibility and usability of the system.

3.6.1. Data Import and Domain Definition

After accessing the app, the upload section is shown (Figure 3.3.). The user begins by uploading a spectral dataset, which contains multiple samples of discretized temperature curves across a 365-day domain. The platform automatically summarizes the data (number of samples, number of points, presence/absence of headers) and displays a preview table (Figure 3.4.).

- At this point, the user defines the functional domain, e.g., from 1 to 365 days, and selects whether the sampling is regular or irregular.
- A validation step ensures consistency between the uploaded file and the chosen configuration (Figure 3.5.).

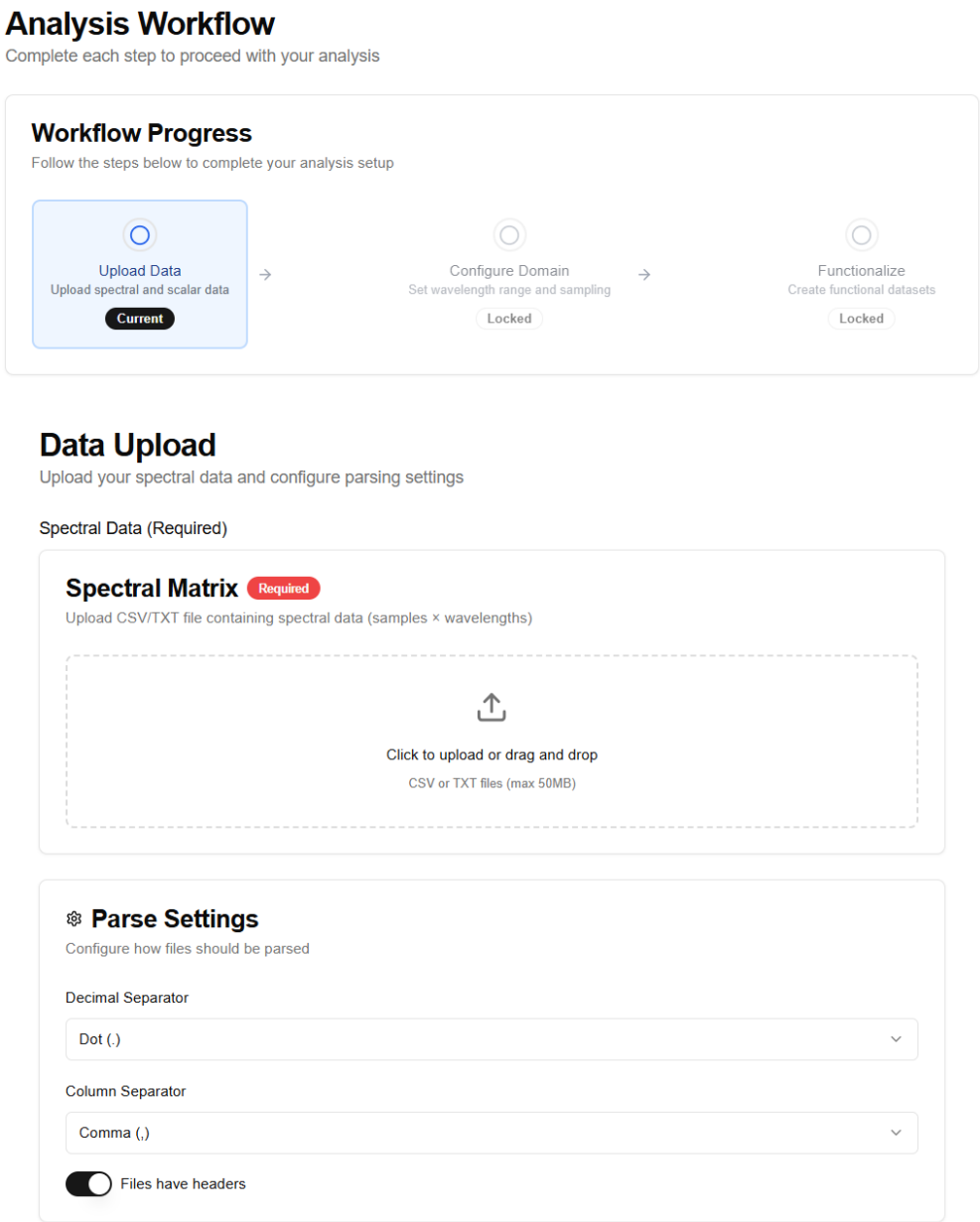

Figure. 3.3. SCREENSHOT BEFORE DATA UPLOAD.

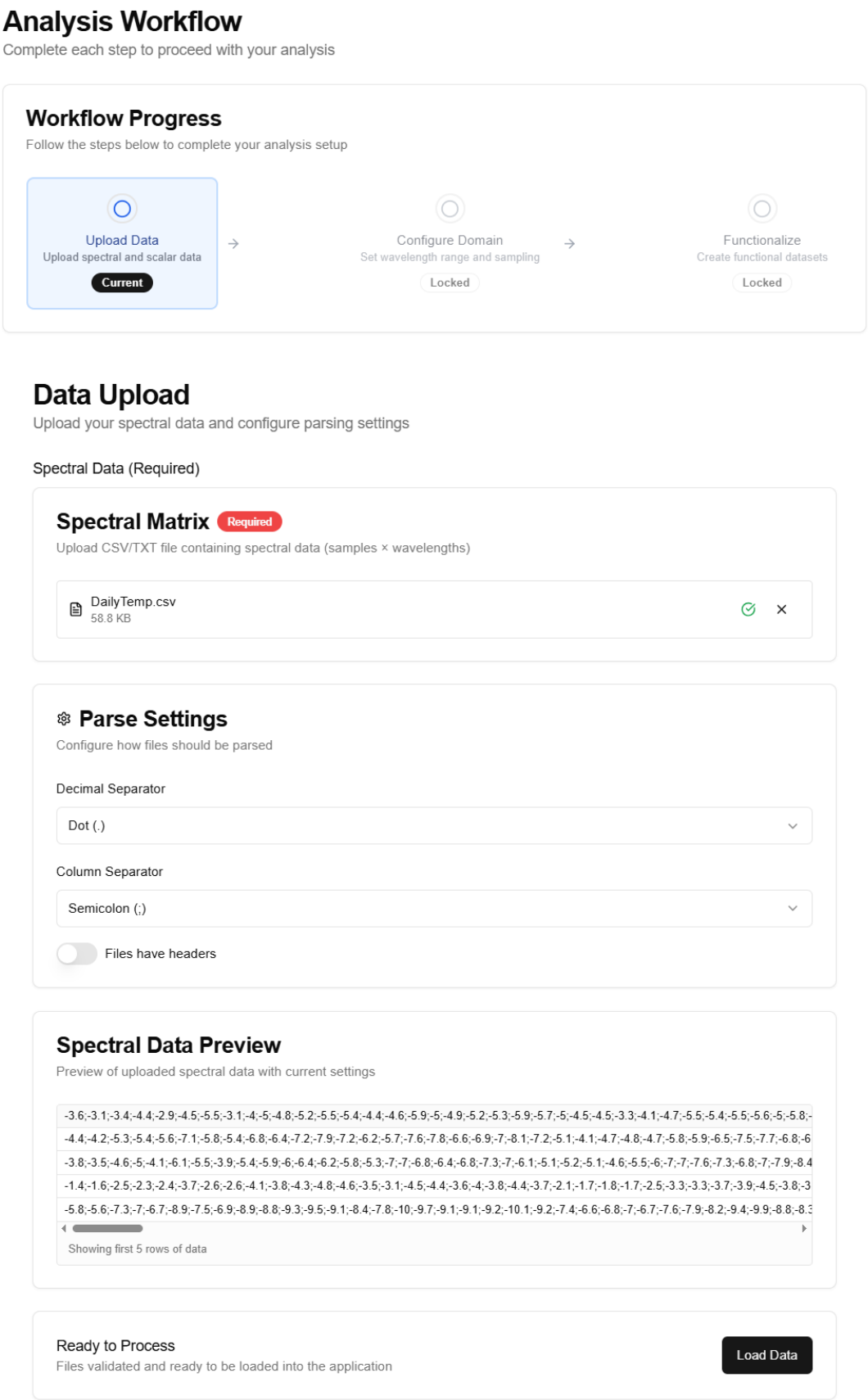

Figure. 3.4. SCREENSHOT AFTER DATA UPLOAD SHOWING SUMMARY AND FEATURES OF THE DATASET.

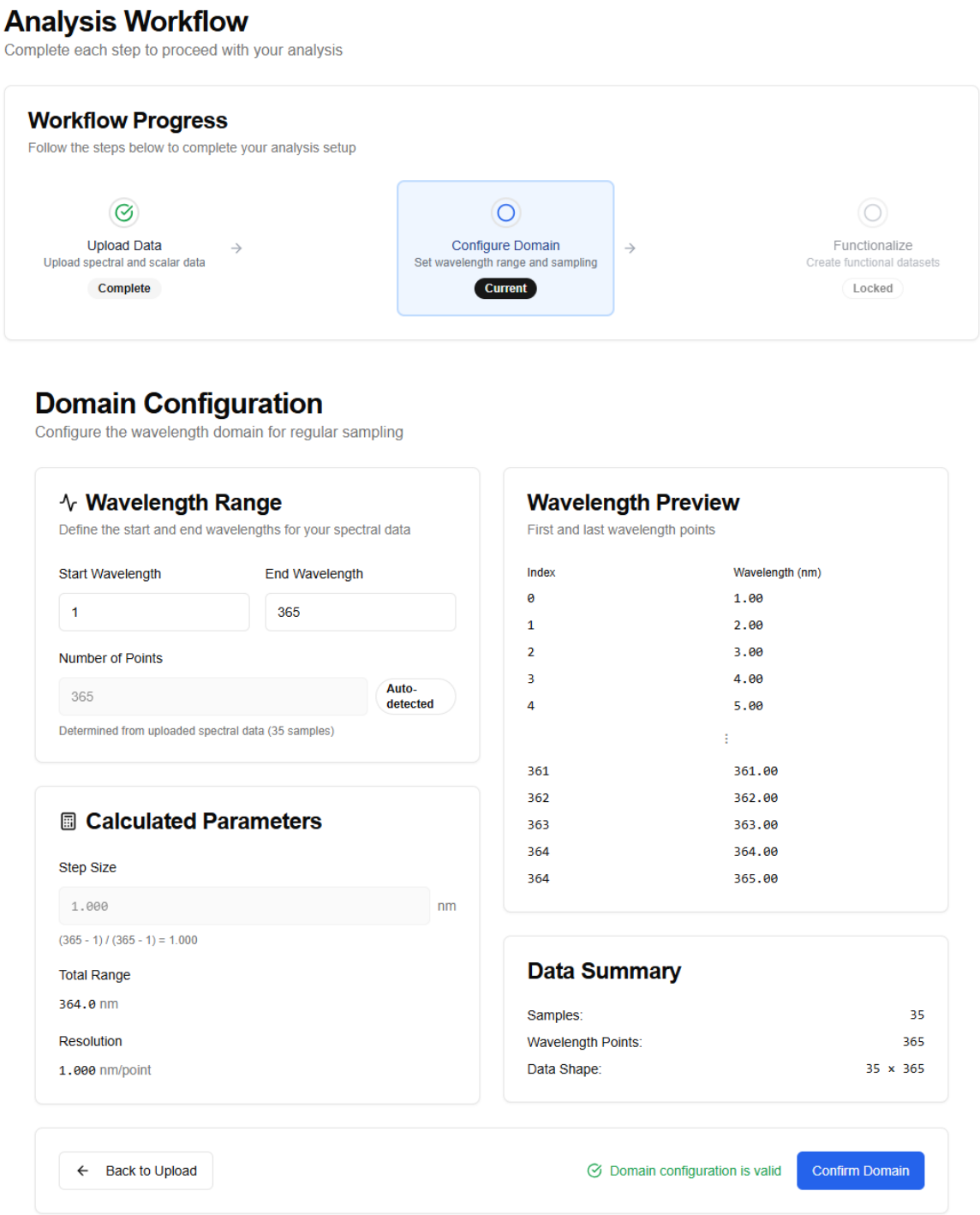

Figure. 3.5. SCREENSHOT OF THE DATA UPLOAD AND DOMAIN SETUP INTERFACE WITH DATASET PREVIEW.

3.6.2 Functionalization

The next step is to convert the raw discrete measurements into smooth functional objects. The user chooses a basis expansion method, such as cubic B-splines with 35 basis functions, and sets the smoothing parameter λ (e.g. 0.25).

- The Auto (GCV) button allows the platform to automatically compute an optimal λ value using generalized cross-validation.

- Once functionalized, the curves are displayed interactively, with the possibility of zooming and comparing them across the domain (Figure 3.6.).

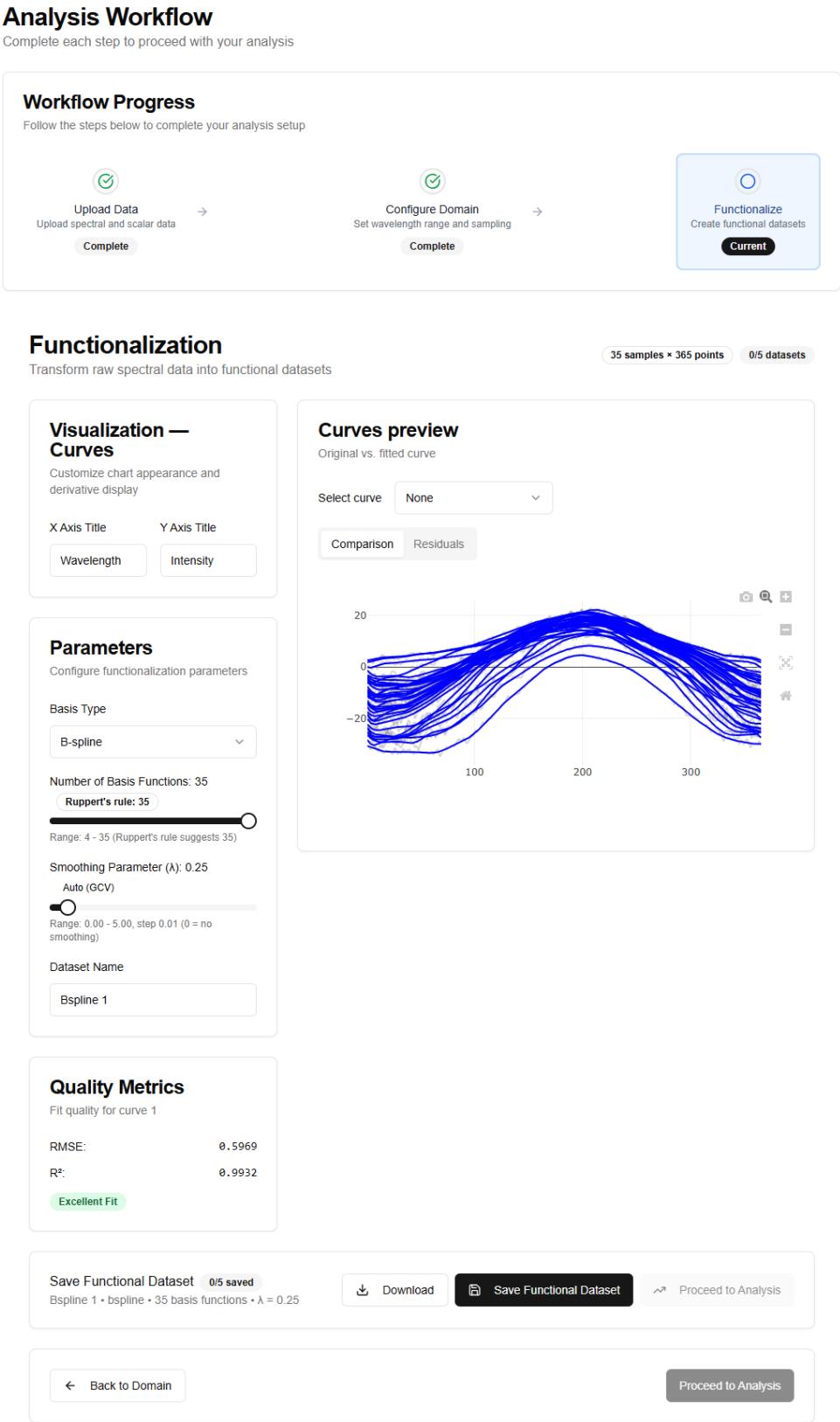

Figure 3.6. SCREENSHOT OF THE FUNCTIONALIZATION INTERFACE SHOWING SMOOTHED CURVES WITH CHOSEN BASIS AND λ.

Once everything is ready, the user must save at least one dataset for being able to proceed to the analysis dashboard (Figure 3.7.).

Figure 3.7. SCREENSHOT OF THE FUNCTIONALIZATION INTERFACE AFTER SAVING A DATASET AND ALLOWING THE USER TO PROCEED TO ANALYSIS.

3.6.3 Exploratory Analysis

Before proceeding with modelling, the user explores the functional dataset using descriptive tools:

- Mean function with 95% confidence bands.
- Pointwise variance across the domain.
- Correlation analysis between curves.

These exploratory tools help to identify global patterns, variability regions, and potential group differences (Figure 3.8.).

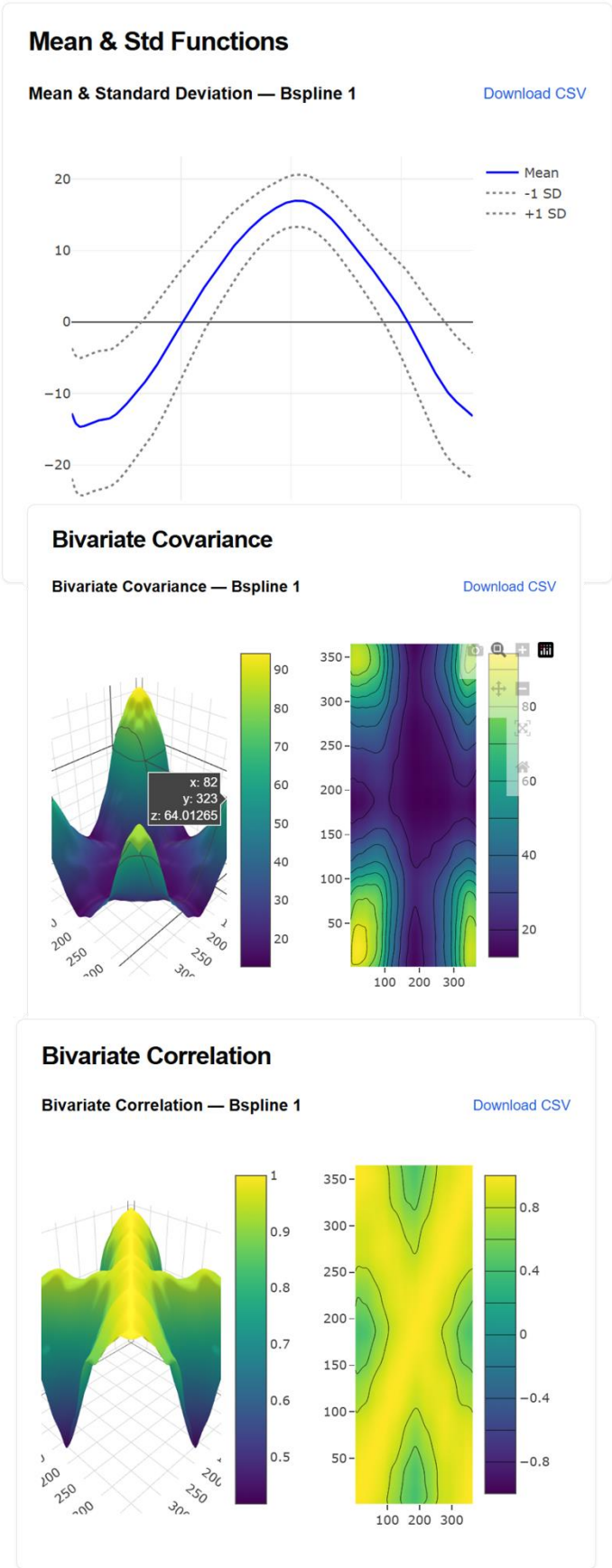

Figure 3.8. SCREENSHOT OF MEAN CURVE WITH CONFIDENCE BANDS AND VARIANCE PLOT.

3.6.4 Unsupervised Analysis: FPCA

Functional Principal Component Analysis (FPCA) is applied to reduce dimensionality and extract the main modes of variation.

- The user selects the number of components (e.g., 3).
- The application displays the eigenfunctions (principal components), their explained variance, and the FPC scores for each sample.

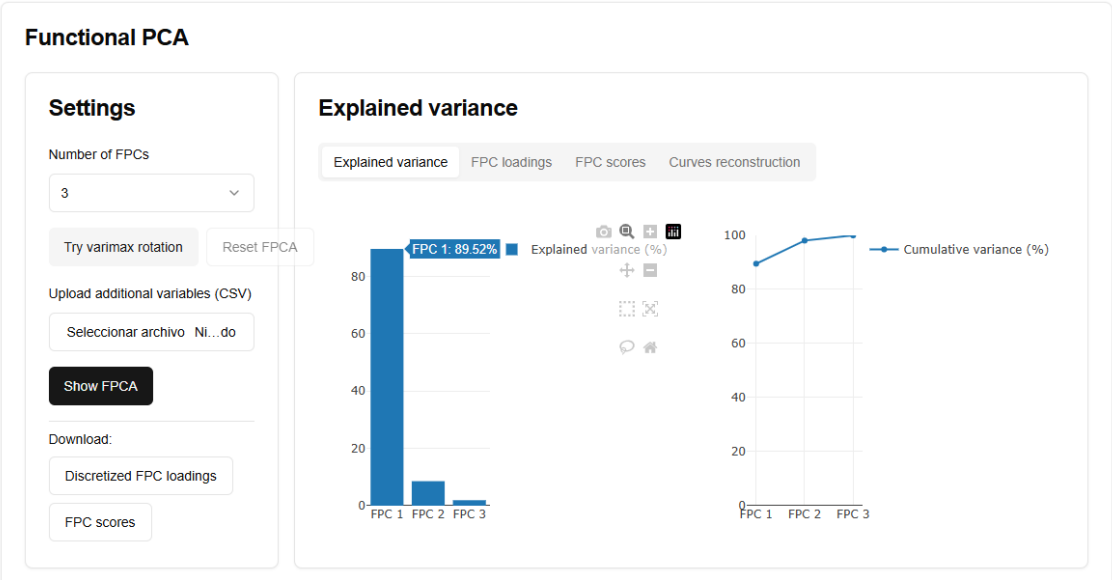

Figure 3.9. SCREENSHOT OF FPCA RESULTS SHOWING EIGENFUNCTIONS AND VARIANCE EXPLAINED.

3.6.5 Supervised Analysis

The platform provides multiple regression and classification techniques:

- SoFR: predicts a scalar response (e.g., concentration levels) using functional predictors.
- FoSR: models functional responses based on scalar predictors.
- FoFR: relates functional predictors and functional responses simultaneously.
- FPCR: performs regression using FPCA scores as predictors.
- FPCLoR: classifies binary outcomes using FPCA scores.
- FLDA: provides supervised classification based on functional features.

Each supervised method generates:

- A model summary (coefficients, R^2/accuracy, statistical tests) (Figure 3.10. for FPCLoR).
- (For classification models) ROC curves and confusion matrices (Figure 3.11. for FPCLoR).

- Fitted values and residuals with interactive plots (Figure 3.12. for FPCLoR).
- Functional parameter β(t) with confidence intervals (Figure 3.13. for FPCLoR).

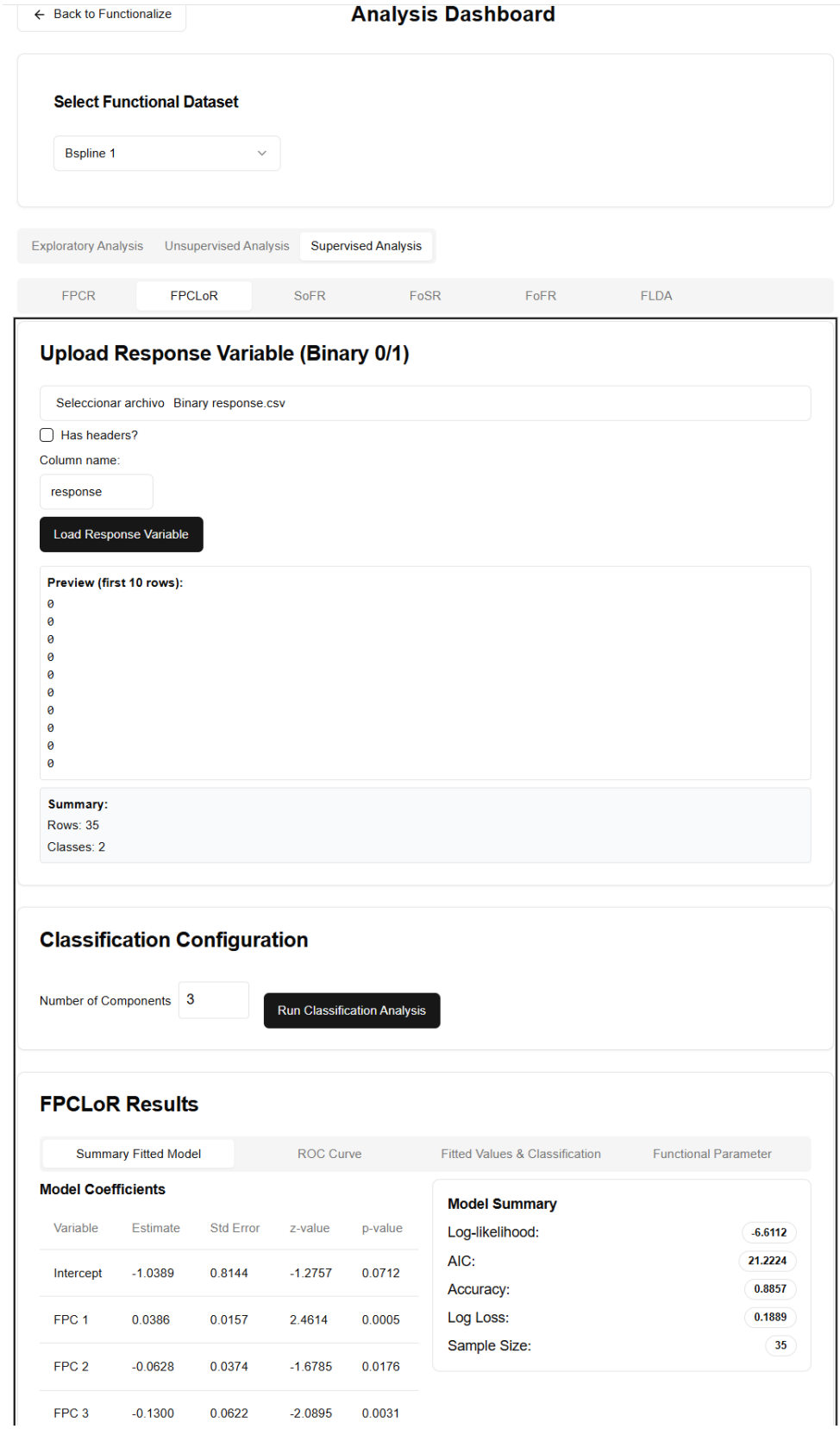

Figure 3.10. SCREENSHOT OF MODEL SUMMARY IN FPCLOR.

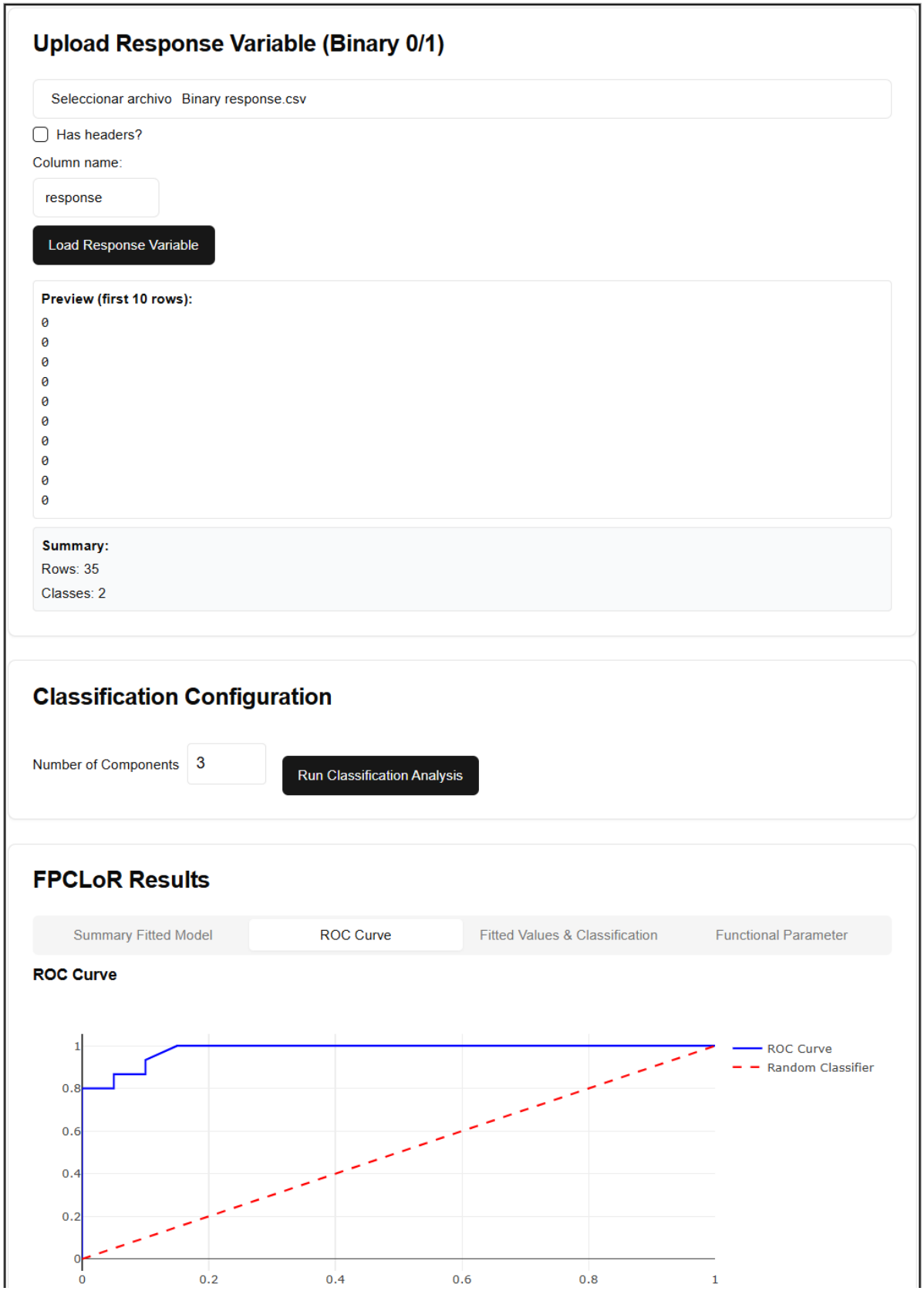

Figure 3.11. SCREENSHOT OF ROC CURVE IN FPCLOR.

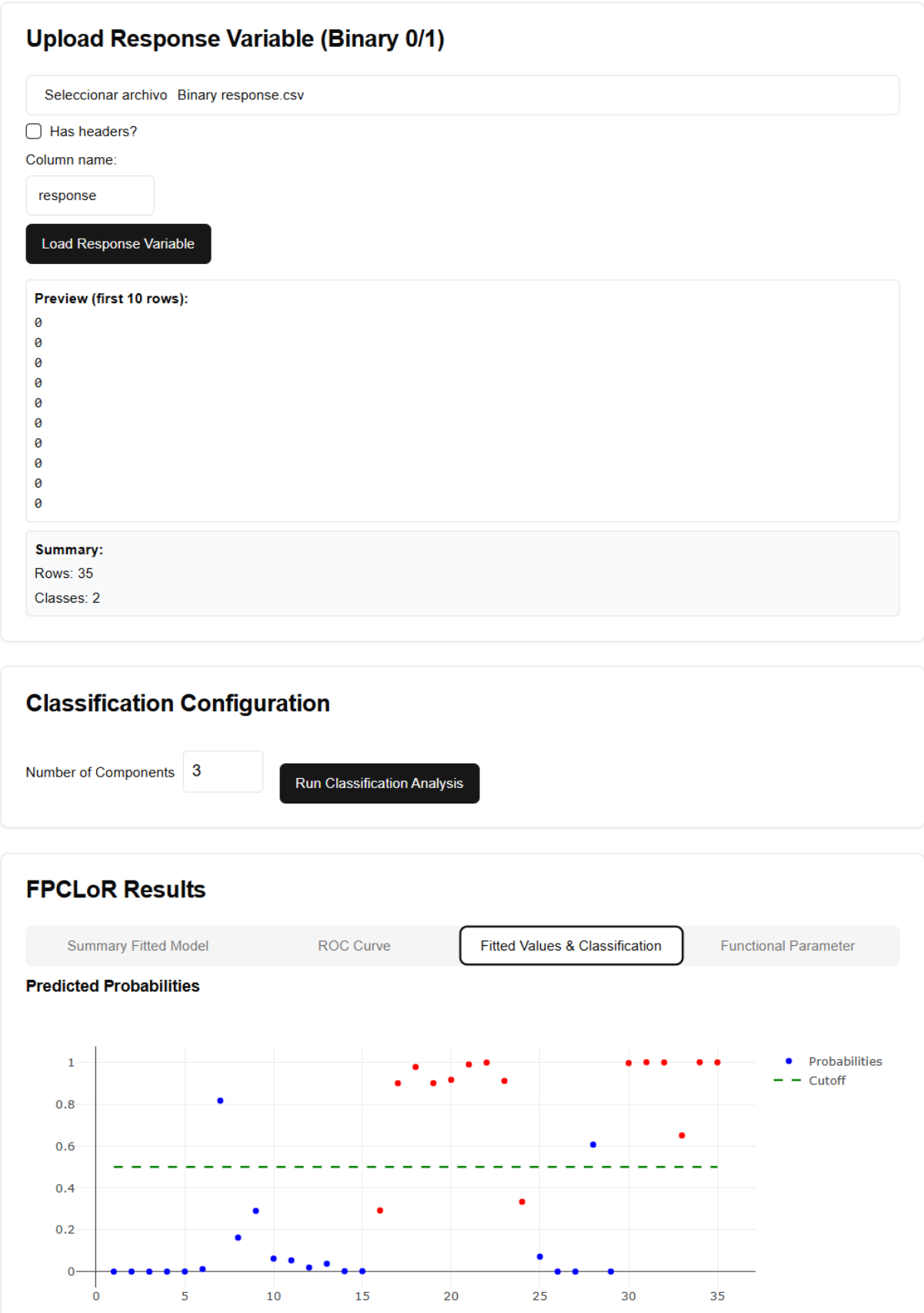

Figure 3.12. SCREENSHOT OF FITTED VALUES AND RESIDUALS IN FPCLOR.

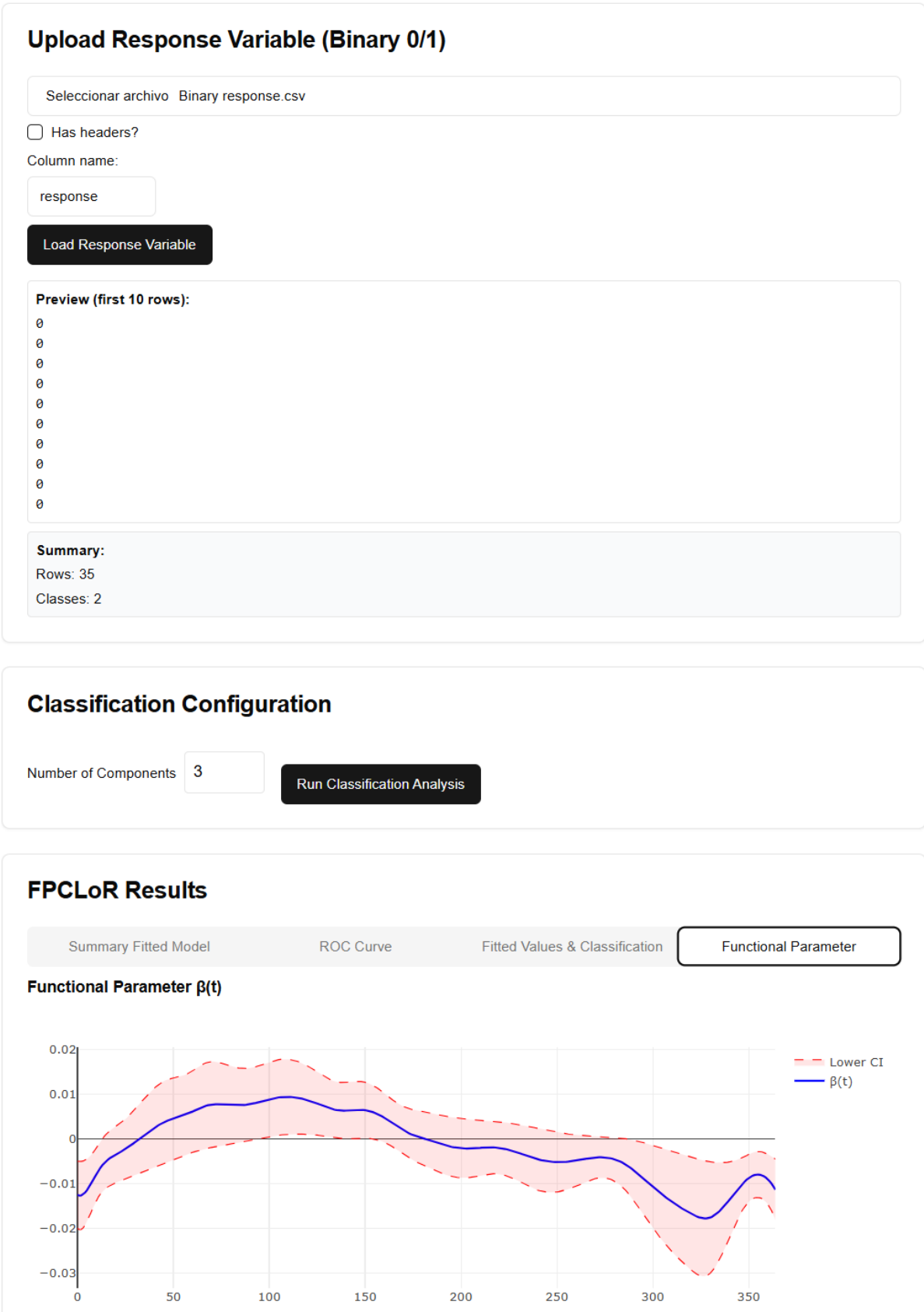

Figure 3.13. SCREENSHOT OF FUNCTIONAL PARAMETERS AND CONFIDENCE INTERVALS IN FPCLOR.

4. RESULTS

This chapter presents how the implemented web application enables a complete Functional Data Analysis (FDA) workflow on spectral datasets. It illustrates the user interface and outputs at each stage, highlighting the interactivity, reproducibility, and usability of the system. The objective is to demonstrate that the platform successfully integrates data preprocessing, exploratory tools, unsupervised methods, and supervised models within a single environment.

4.1. User Workflow: Data Upload and Domain Configuration

The user is guided through a sequential workflow that ensures proper data preparation before any analysis. The steps include:

Upload Data: Import spectral datasets in CSV or TXT format, with configuration options for decimal separator, column separator, and header presence. Optionally, a response dataset can be uploaded for supervised models. A preview table is displayed immediately, and the system validates the input format.

Domain Setup: Define the functional domain (e.g., 1–365 days or 400–700 nm). The system computes the step size automatically and renders a preview of the grid.

Validation: Summaries and status indicators confirm that the uploaded data is consistent and ready for functionalization.

This structured process prevents errors, ensuring that subsequent steps rely on properly formatted data.

4.2. Functional Representation

Raw spectral measurements are transformed into smooth functional data objects using basis expansion methods:

- Cubic B-splines, for flexible local fitting.
- Fourier basis, for periodic data.
- Wavelet basis, for localized variations.

The number of basis functions and smoothing parameter λ are configurable. Additionally, the system integrates GCV to automatically suggest an optimal λ.

Users can compare original and smoothed spectra interactively, visualize error metrics (RMSE, R^2), and export basis coefficients or discretized fitted functions.

4.3. Exploratory Analysis

Once functional data is obtained, the exploratory module provides statistical summaries and interactive plots:

- Mean spectrum with optional confidence bands.

- Pointwise variance curves highlighting variable regions.
- Correlation analysis across spectral points.

These tools help users identify trends, variability sources, and potential discriminant regions before moving to dimensionality reduction or supervised modelling.

4.4. Unsupervised Analysis: Functional Principal Component Analysis

FPCA decomposes functional variability into orthogonal components. The application computes:

- Principal component functions (PCFs).
- Explained variance and cumulative variance plots.
- Component scores for each sample.

The user selects the number of components, with optional varimax rotation for improved interpretability. FPCA is also the foundation for supervised models such as FPCR and FPCLoR.

4.5. Supervised Analysis

The platform integrates a wide set of supervised techniques, supporting both regression and classification:

- SoFR: scalar response predicted from functional curves.
- FoSR: functional response modelled using scalar predictors.
- FoFR: both predictors and response are functional.
- FPCR: regression using FPCA scores as predictors.
- FPCLoR: logistic regression on FPCA scores for binary classification.
- FLDA: classification using functional predictors and discriminant functions.

Each method outputs:

- Model summary (coefficients, metrics, statistical significance).
- Interactive plots of fitted values and residuals.
- Functional parameter $\beta(t)$ with 95% confidence intervals.
- For classification: confusion matrices and ROC curves.

4.6. Export and Reproducibility

At every stage, the user can download intermediate and final results:

- Functional representation: basis coefficients, discretized functions.
- FPCA: loadings, scores, explained variance.

- Supervised models: fitted values, predictions, residuals, functional β(t).
- Visualizations: PNG export for all plots.

These export features ensure full reproducibility and facilitate the inclusion of results in scientific reports. Table 4.1 summarizes available download formats.

Table 4.1. SUMMARY OF DOWNLOAD FORMATS

Content	Format
Fitted functional data	CSV
Coefficient matrix	CSV
FPCA scores	CSV
Model predictions	CSV
Plots and visualizations	PNG

4.7. Summary of Functionality

The developed platform successfully integrates functional representation, exploratory tools, dimensionality reduction, and supervised models into a single, interactive system. Key strengths include:

- Comprehensiveness: covers the full FDA pipeline, from raw data upload to supervised modelling.
- Accessibility: web-based interface requiring no programming skills.
- Reproducibility: download options for all results and parameters.
- Interactivity: dynamic plots and configurable parameters.
- Extensibility: modular design enabling future integration of additional FDA techniques.

Table 4.2 summarizes core modules and functionalities.

Table 4.2. SUMMARY OF CORE MODULES AND FEATURES IMPLEMENTED

Module	Key Functionalities
Upload	Supports CSV/TXT, decimal/column separators
Domain Configuration	Auto-detects ranges, allows manual override
Functionalization	B-spline, Fourier, Wavelet basis + smoothing, GCV, Ruppert
Exploratory Analysis	Interactive plots (mean, SD, correlation, histogram)
Unsupervised	Variance explained, PCFs, Varimax, score plots
Supervised	Regression/classification from FPCA components
Export System	Curve data, coefficients, metrics, plots (CSV, PNG)

5. DISCUSSION AND CONCLUSIONS

5.1. General Conclusions

The design and implementation of this web-based platform for FDA represents a meaningful step forward in making advanced statistical tools accessible to a wider scientific audience, which aligns with the theoretical foundations outlined in prominent reviews of FDA [27]. By integrating robust mathematical foundations with state-of-the-art frontend and backend technologies, the application bridges the gap between theoretical FDA methodologies and their practical application in spectral data contexts.

The platform successfully combines a modular architecture with dynamic, interactive user experiences. It offers:

- Smooth workflows for uploading and preprocessing datasets
- High-quality, publication-ready visualizations
- Flexible functionalization via B-splines, Fourier, and Wavelets
- Core FDA methods such as exploratory analysis, unsupervised and supervised techniques.
- Export and persistence of results for reproducibility

These functionalities are accessible through a user interface designed for non-experts in statistics or programming, making this tool especially valuable for applied researchers, data science students, and interdisciplinary teams working with continuously recorded data.

5.2. Evaluation of Objectives

The core objectives outlined at the beginning of the project have been met successfully:

Table 5.1. SUMMARY OF ACHIEVED OBJECTIVES

Achieved Objectives
Build a modular, interactive web interface using modern tools
Enable data upload and domain configuration with flexibility
Implement functionalization with B-splines, Fourier, and Wavelet bases
Provide smoothing options (manual and automatic via GCV)
Incorporate dimensionality reduction through FPCA
Implement supervised models: FPCR and FPCLoR
Ensure exportability and reproducibility of all results

Each module has been validated through test datasets, and their performance confirmed visually and quantitatively (e.g., RMSE, R^2).

5.3. Limitations

Despite achieving its foundational goals, the application still has room for improvement in various areas:

- Irregular sampling: The system currently assumes regularly spaced observations; irregular domain support is partially implemented but not exposed in the UI.
- Interpretability tools: Beyond metrics like RMSE and R^2, the platform lacks diagnostic plots such as residual distributions, variable importance, or contribution scores. These aspects are particularly important in high-stakes biomedical applications, where interpretability remains a core requirement [19].
- Deployment model: The app runs locally; it does not yet support cloud-based execution, session sharing, or user authentication.

These constraints are technical rather than conceptual and are considered low to medium complexity for future development.

5.4. Future Lines of Work

Several future directions can further extend the capabilities of this platform:

- Multivariate Data Analysis: A key expansion is to allow full support for multivariate data workflows (e.g., PCA, PLS, LDA, K-means) with dynamic switching between functional and multivariate modes, enabling integrated comparative analysis.
- Support for Irregular Domains: Extend the preprocessing and functionalization modules to support irregularly spaced observations and domain warping techniques.
- Advanced Model Diagnostics: Include tools for inspecting model performance such as residual plots, confidence intervals, biplots, and variable importance metrics.
- User-Centric Features: Develop session saving, project exporting, multi-user collaboration, and backend persistence using databases.
- Cloud Deployment: Enable server-side execution and cloud hosting for real-time collaborative usage, using containerization (Docker) and deployment platforms (e.g., Heroku, Vercel, or AWS).
- Educational Mode: Include a tutorial mode with guided examples and explanations to support teaching FDA in academic environments.
- Automat-ML Capabilities: Explore integration of automated model selection and parameter tuning for supervised models to improve usability for non-experts.

This roadmap ensures the project remains scalable and adaptable for broader research, industrial, and educational use cases in data science and engineering.

6. REGULATORY FRAMEWORK

6.1. Data Privacy and Protection

Although the current implementation of the application is intended for local use and does not store any data on external servers, it is important to anticipate that some users might upload sensitive information. This is especially relevant in research domains involving biomedical or clinical samples, where functional data may include biometric patterns (e.g., spectral curves from human or animal samples). Such spectral approaches have been employed in non-invasive biomedical sensing, reinforcing the need for strict data governance [28].

In compliance with the General Data Protection Regulation (GDPR – EU Regulation 2016/679), the following measures are considered essential:

- Users must anonymize any personal or identifiable data before uploading it to the platform.
- The application does not persistently store uploaded data, unless explicitly saved by the user for analysis. All temporary data is handled in volatile memory.
- It is recommended to include warning messages in the interface, notifying users when potentially sensitive data is detected or when attempting to download/export derived results from such data.

Although the local nature of the application reduces the risk of data exposure, this risk is not eliminated. Therefore, it is essential to inform users that they remain responsible for any personal data they decide to process using this tool.

6.2. Ethical and Professional Responsibility

As the developer of a tool that could influence experimental or even diagnostic decisions, the responsibility for ethical and professional practices in software design must be acknowledged. Several safeguards have been considered:

- The application is intended strictly for research and educational purposes. Any result obtained must be interpreted by qualified professionals and not used as standalone diagnostic evidence.
- The software includes capabilities to log configuration parameters, export results in standard formats (CSV, JSON), and reproduce previous analyses, fostering transparency and reproducibility.
- Users are reminded through disclaimers that the models and visual outputs generated by the app may be limited by the quality of the input data and by the assumptions underlying each statistical method.

- Where predictive models are applied (e.g., classification via FPCA), it is important to highlight limitations and avoid misleading confidence in model output.

Future versions of the app may include a dedicated documentation section or tutorial highlighting the ethical use of analytical tools, especially when applied to real-world biomedical problems.

6.3. Technical Standards and Good Practices

The development of the application followed widely adopted technical standards and software engineering practices to ensure code quality, maintainability, and future scalability. The main principles are summarized below:

- Technologies Used: Python was chosen due to its extensive scientific ecosystem and open-source nature. Key libraries include:
    - scikit-fda for functional data analysis.
    - plotly for interactive data visualization.
    - numpy and pandas for data manipulation.
    - FastAPI and Next.js for backend and frontend architecture.
- Rationale for Selection:
    - All tools have active communities, through documentation, and are interoperable with other platforms.
    - Open-source licensing supports reproducibility and facilitates peer validation.
- Best Practices Adopted:
    - Modular architecture following separation of concerns.
    - Version control using Git for collaborative development.
    - Code comments and documentation to ensure future maintainability.

These practices not only improve software quality but also foster long-term sustainability, especially within academic environments such as the SITec research group.

6.4. Intellectual Property and Licensing Strategy

This project has been developed with the goal of contributing to the open science movement and to the toolset of the SITec research group. The intellectual property strategy has been guided by the following principles:

- Copyright Transfer: As the author, I formally agree to transfer the rights of the developed code and accompanying materials to the SITec research group at UC3M, allowing reuse, extension, and adaptation in future research projects.

- Open-Source Licensing: The application will be released under the MIT License, a widely recognized permissive license that:
    - Allows unrestricted use, modification, and distribution.
    - Supports both academic and non-commercial applications.
    - It is compatible with the licenses of third-party libraries used.
- Acknowledgment of Dependencies: All external libraries used in the application are compatible with MIT licensing. A LICENSE file will be included in the repository, along with a Credits section recognizing major contributors and dependencies.

This licensing choice supports both transparency and collaboration, ensuring that future developers, researchers, or students can build upon the tool while respecting the terms of reuse.

6.5. Concluding Remarks on the Regulatory Framework

From the beginning of the project, regulatory, ethical, and technical aspects have been embedded into the development process. These considerations strengthen the scientific and social value of the application, ensuring that:

- The tool can be responsibly used in a variety of research settings.
- Its design aligns with the values of open science, reproducibility, and user empowerment.
- It can evolve into a collaborative platform that supports not only data analysis but also education, transparency, and interdisciplinary cooperation.

By anticipating challenges related to privacy, ethics, and licensing, this work contributes not only to functional products, but also a responsible and forward-thinking digital research infrastructure.

7. SOCIO-ECONOMIC ENVIRONMENT

7.1. Impact on Research and Technology Transfer

The developed application has a direct impact on scientific and technological research by enabling advanced statistical methods for the analysis of spectral and functional data. This contribution supports:

- Increased efficiency in experimental data processing, allowing faster and more intuitive exploratory analysis.
- Greater autonomy for early-stage researchers (e.g., master's and doctoral students), who can now apply complex techniques without needing advanced programming skills.
- Open scientific tool reuse, encouraging knowledge sharing among institutions, research groups, and technology-based enterprises.

By offering open-source access, the application reduces dependence on expensive proprietary licenses and lowers the technological barrier for institutions with limited resources. This directly promotes equity in science and strengthens research capacity in underfunded academic or scientific environments.

7.2. Contribution to Scientific and Technical Education

Although the main goal of the platform is not educational, the application has clear pedagogical value in workshops and training sessions (e.g., Marie Skłodowska-Curie networks or summer schools), where early-career researchers learn how to explore and analyze real-world functional datasets.

Its interactive interface, visual outputs, and modular design can serve as a bridge between theoretical statistics and empirical intuition. By interacting with concepts such as curve fitting, basis decomposition, and dimensionality reduction, students can develop a deeper understanding of statistical modelling in functional spaces, even if they lack formal training in computer science or mathematics.

Ultimately, tools like this promote accessible statistical education, especially in interdisciplinary academic environments where users often come from biology, chemistry, engineering, or environmental sciences.

7.3. Potential Impact on Applied Sectors

While the tool is primarily designed for academic and research purposes, it has indirect applicability in various industrial sectors:

- Biomedical research: for non-invasive spectral diagnostics and monitoring, including potential use in early detection of disease markers, as shown in previous studies using mm-wave spectroscopy for early detection [29].

- Agro-food and environmental industries: for quality control or detection of contaminants through portable spectroscopic devices.
- Emerging technologies: such as intelligent sensors and embedded spectroscopic systems, where functional data analysis is key for device calibration and system validation.

By providing a flexible, intuitive, and open framework, this tool could benefit small and medium-sized enterprises (SMEs), R&D departments, and innovation-oriented startups that seek low-cost solutions for exploratory analysis and prototyping.

7.4. Alignment with the Sustainable Development Goals (SDGs)

The project aligns with several of the United Nations Sustainable Development Goals (SDGs). Among them:

- SDG 4. Quality Education: By offering an open, hands-on tool for training in functional data analysis, the platform enhances accessibility to advanced analytical techniques and supports the education of researchers and students.
- SDG 9. Industry, Innovation, and Infrastructure: The use of open-source technologies, combined with a modular and extensible architecture, fosters innovation and promotes technological self-sufficiency in research institutions.
- SDG 10. Reduced Inequalities: By lowering the cost and technical barriers to advanced data analysis, this application helps democratize access to state-of-the-art scientific methods, particularly in low-resource settings. FDA has already demonstrated its applicability in domains such as clinical monitoring and medicine, where access to interpretative tools is vital [30].

Other indirect contributions could be made to SDG – 3 Good Health and Well-Being, through its potential use in biomedical data analysis, and to SDG 17 – Partnerships for the Goals, by supporting collaboration and resource sharing between research groups, institutions, and networks.

7.5. Estimated Budget and Costs

In order to complement the socio-economic perspective, it is important to provide an estimate of the resources required to develop and deploy the application. The main cost factor is the human effort invested in the development of the backend (Python modules for functional data analysis) and the frontend (Next.js/React interface). Additionally, a basic computer server is required to run the platform locally, but no further infrastructure is needed.

Thanks to the exclusive use of open-source software packages (e.g., Next.js, React, Tailwind, Plotly, scikit-fda, NumPy, Pandas), licensing costs are reduced to **zero**, which makes the project highly cost-efficient.

Table 7.1 summarizes the estimated costs:

Table 7.1. ESTIMATED DEVELOPMENT AND DEPLOYMENT COSTS

Item	Estimated Hours	Cost per Hour (€)	Total Cost (€)
Backend development (Python, FDA methods, exports)	180	25 €/h	4500 €
Frontend development (Next.js, React, visualizations, UI)	160	25 €/h	4000 €
Local computer/server (standard workstation)	-	-	1200 €
Software and libraries (open-source)	-	-	0 €
Total	**≈ 340 h**	**-**	**5700 €**

These estimates assume a standard academic rate of **25 €/h**, commonly used for R&D project valuations. The total development time is capped at 360 h, divided almost evenly between frontend and backend, ensuring feasibility within a single research project. The only hardware requirement is a standard workstation, which may also be reused for other research tasks.

Overall, the project demonstrates **high economic efficiency**, since for a relatively small investment (≈ 5,700 €), a complete functional data analysis platform has been developed, with direct applicability in research, education, and potential industrial contexts.

7.6. Final Considerations

Even though this project was developed within the scope of a final year thesis, it promotes key values in contemporary science: accessibility, transparency, openness, and efficiency. These values extend beyond technical implementation, fostering a broader impact in research practices, scientific education, and innovative ecosystems.

By enabling intuitive exploration of functional data and supporting reproducible workflows, this tool has the potential to become a useful asset not only for the SITec research group, but also for the broader scientific community.

8. BIBLIOGRAPHY

[1] C. A. Lieber and A. Mahadevan-Jansen, "Automated Method for Subtraction of Fluorescence from Biological Raman Spectra," Applied Spectroscopy, vol. 57, no. 11, pp. 1363–1367, Nov. 2003, doi: 10.1366/000370203322554518.

[2] F. Malongane, L. J. McGaw, H. Nyoni, and F. N. Mudau, "Metabolic profiling of four South African herbal teas using high resolution liquid chromatography-mass spectrometry and nuclear magnetic resonance," Food Chemistry, vol. 257, pp. 90–100, Aug. 2018, doi: 10.1016/j.foodchem.2018.02.121.

[3] G. O. Ariunbold, A. Byopadhyay, K. Parameswaran, J. Sacher, and A. Sengupta, "Advanced Spectroscopy in Precision Agriculture," Optics and Photonics News, vol. 30, no. 11, p. 40, Nov. 2019, doi: 10.1364/OPN.30.11.000040.

[4] C. Chen et al., "Stand-off hazardous materials identification based on near-infrared hyperspectral imaging combined with convolutional neural network," Spectrochimica Acta Part A: Molecular and Biomolecular Spectroscopy, vol. 327, p. 125311, Feb. 2025, doi: 10.1016/j.saa.2024.125311.

[6] J. O. Ramsay and B. W. Silverman, "Functional Data Analysis," in Functional Data Analysis, Second., Wiley, 2005. doi: 10.1002/0470013192.bsa239.

[8] F. Ferraty and P. Vieu, Nonparametric Functional Data Analysis: Theory and Practice. Springer, 2006. [Online]. Available in: https://www.researchgate.net/publication/237131107_Nonparametric_Functional_Data_Analysis_Theory_and_Practice

[9] M. M. Ahsan, S. A. Luna, and Z. Siddique, "Machine-Learning-Based Disease Diagnosis: A Comprehensive Review," Healthcare, vol. 10, no. 3, p. 541, Mar. 2022, doi: 10.3390/healthcare10030541.

[5] I. T. Jolliffe and J. Cadima, "Principal component analysis: a review and recent developments," Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences, vol. 374, no. 2065, Apr. 2016, doi: 10.1098/rsta.2015.0202.

[7] C. M. . Bishop, Pattern recognition and machine learning. Springer Science Business Media, 2006. [Online]. Available in: https://www.microsoft.com/en-us/research/wp-content/uploads/2006/01/Bishop-Pattern-Recognition-and-Machine-Learning-2006.pdf

[10] MathWorks, "MATLAB Documentation," [Online]. Available in: https://www.mathworks.com/products/matlab.html

[11] R Core Team, "R: A language and environment for statistical computing." R Foundation for Statistical Computing, 2024. [Online]. Available in: https://www.r-project.org

[12] Python Software Foundation, “Python Programming Language.” Python Software Foundation, 2024. [Online]. Available in: https://www.python.org

[13] M. Febrero-Bande and M. O. de la Fuente, “Statistical Computing in Functional Data Analysis: The R Package fda.usc,” Journal of Statistical Software, vol. 51, no. 4, 2012, doi: 10.18637/jss.v051.i04.

[14] Y. Zhu, “Functional Data Analysis of Spectroscopic Data with Application to Classification of Colon Polyps,” American Journal of Analytical Chemistry, vol. 08, no. 04, pp. 294–305, 2017, doi: 10.4236/ajac.2017.84022.

[15] C. Ramos-Carreño, J. L. Torrecilla, M. Carbajo-Berrocal, P. Marcos, and A. Suárez, “scikit-fda: A Python Package for Functional Data Analysis,” Journal of Statistical Software, vol. 109, no. 2, 2024, doi: 10.18637/jss.v109.i02.

[16] Steven Golovkine, “FDApy: a Python package for functional data,” Jan. 2021.

[17] STATFDA, “STATFDA: Web Application for Functional Data Analysis,” [Online]. Available in: https://www.statfda.org.

[18] A. Moreno-Oyervides et al., “Early, Non-Invasive Sensing of Sustained Hyperglycemia in Mice Using Millimeter-Wave Spectroscopy,” Sensors, vol. 19, no. 15, p. 3347, Jul. 2019, doi: 10.3390/s19153347.

[19] A. Cuevas, “A partial overview of the theory of statistics with functional data,” Journal of Statistical Planning and Inference, vol. 147, pp. 1–23, Apr. 2014, doi: 10.1016/j.jspi.2013.04.002.

[20] Manoj Satishkumar, “Frontend Frameworks Comparison 2025,” manojsatishkumar, 2025. [Online]. Available in: https://manojsatishkumar.com/blog/frontend-frameworks-comparison-2025.html

[21] Huynh Trong Hieu, “Top Frontend Frameworks of 2025 Ranked by GitHub Stars,” NashTechGlobal, Jun. 2025. [Online]. Available in: https://blog.nashtechglobal.com/top-frontend-frameworks-in-2025/

[22] Vikas Singh, “Top JavaScript Frameworks Comparison for Web Development in 2025,” Brilworks, Jun. 2025. [Online]. Available in: https://www.brilworks.com/blog/javascript-web-frameworks-comparison/

[23] Stack Overflow, “2024 Developer Survey,” survey.stackoverflow, 2024. [Online]. Available in: https://survey.stackoverflow.co/2024/technology#2-programming-scripting-and-markup-languages

[24] Jesús López, “Main differences between matplotlib, seaborn, and plotly,” Medium, May 2024. [Online]. Available in: https://medium.com/datons/main-differences-between-matplotlib-seaborn-and-plotly-709307e7e00b

[25] Scikit-fda Development Team, “Scikit-fda Documentation,” 2024. [Online]. Available: https://fda.readthedocs.io

[26] FDApy Project, “FDApy Documentation,” 2024. [Online]. Available: https://fdapy.github.io

[27] Hans-Georg Müller and Barbara E. Hall, “Review of functional data analysis,” Annual Review of Statistics and Its Application, vol. 2, no. 1, pp. 321–359, Mar. 2015, doi: 10.1146/annurev-statistics-010814-020413.

[28] A. Moreno-Oyervides, M. C. Aguilera-Morillo, F. Larcher, V. Krozer, and P. Acedo, “Advanced Statistical Techniques for Noninvasive Hyperglycemic States Detection in Mice Using Millimeter-Wave Spectroscopy,” IEEE Transactions on Terahertz Science and Technology, vol. 10, no. 3, pp. 237–245, May 2020, doi: 10.1109/TTHZ.2020.2967236.

[29] P. Martín-Mateos et al., “In-vivo, non-invasive detection of hyperglycemic states in animal models using mm-wave spectroscopy,” Scientific Reports, vol. 6, no. 1, p. 34035, Sep. 2016, doi: 10.1038/srep34035.

[30] N. Orozco, S. Ortiz, and G. A. Ospina-Tascón, “Functional Data Analysis Applications in Medicine: A Systematic Review,” WIREs Computational Statistics, vol. 17, no. 2, Jun. 2025, doi: 10.1002/wics.70026.

9. ANNEX A – DECLARATION OF GENERATIVE AI USE

DECLARATION OF USE OF GENERATIVE ARTIFICIAL INTELLIGENCE (AI) IN BACHELOR THESIS (TFG)

I have used Generative AI in my TFG

Check all that apply:

YES	NO
☒	

If you have ticked YES, please complete the following 3 parts of this document:

Part 1: Reflection on legal, ethical and responsible behavior

Please be aware that the use of AI carries some risks and may lead to a series of serious academic consequences: the TFG will not be evaluated by the University if the use of AI involves the use of confidential data, copyrighted materials, or personal data, used without complying with the conditions required in each case (authorization of the interested parties, authorization of the owners, following the instructions of the University).

Question	
1. In my interactions with AI tools, I have provided **confidential data**, always with the appropriate authorization of the data subjects. Confidentiality encompasses any information that a person or organization wishes to protect for legal, commercial, privacy, or strategic reasons (such as patents or trade secrets).	
YES, I have used this data with permission	☒ NO, I have not used confidential data
2. In my interaction with Generative AI tools, I have submitted **copyrighted materials** with the permission of those concerned.	
YES, I have used these materials with permission from the copyright holders, or without permission because they fall within one of the exceptions or limitations permitted by law: • Public domain work	☒ NO, I have not used copyrighted materials

• Licensed work (Creative Commons licenses) • Use of excerpts for research purposes (fair use)	
3. In my interaction with Generative AI tools, I have submitted **personal data** with the consent of the data subjects.	
YES, I have used this data with the authorization of the interested parties and in accordance with the instructions contained in the guide approved by the University.	☒ NO, I have not used personal data
4. My use of the Generative AI tool has **respected its terms of use**, as well as the essential ethical principles, not being maliciously oriented to obtain an inappropriate result for the work presented, that is to say, one that produces an impression or knowledge contrary to the reality of the results obtained, that supplants my own work or that could harm people.	
☒ YES	NO

Part 2: Declaration of technical use

Use the following model statement as many times as necessary, in order to reflect all types of iteration you have had with Generative AI tools. Include one example for each type of use where indicated: *[Add an example]*.

I declare that I have made use of the Generative AI system Perplexity for:

Documentation and drafting:

- *Supporting reflection in relation to the development of the work: iterative process of analysing alternatives and approaches using AI*

I requested sources and references for discussions/debates about programming tools/frameworks to speed up the decision-making process regarding the most recommended tools according to the intended purpose of the web application to be developed; for example, when selecting tools for the frontend and backend.

- *Revision or rewriting of previously drafted paragraphs*

[Add an example] For example: I have asked for paragraphs to be rewritten to shorten the conclusions while keeping the same ideas already written.

- *Search for information or answers to specific questions*

The questions I asked were specific searches about functions in programming packages that could perform the required functionality in the context of developing the application; for example, which function I could use within the scikit-fda package to perform FPCA.

- *Bibliography search*
- *Summary of bibliography consulted*
- *Translation of texts consulted*

Develop specific content

Generative AI has been used as a support tool for the development of the specific content of the TFG, including:

- *Assistance in the development of lines of code (programming)*

I asked for advice on using functions from code packages in Java and Python, such as whether to use Plotly for graphics or the built-in visualization from the scikit-fda package.

- *Generation of diagrams, images, audios or videos*

[indicate its use in this section, as well as in a footnote to the figure/element generated by Generative AI] For example: Source: own elaboration with the help of Midjourney.

- *Optimization processes*

I sought help to find forums (such as Stack Overflow) where errors similar to those I was encountering during development were being discussed.

- *Data processing: collection, analysis, cross-checking of data...*

[Add an example] For example: after collecting the data, I have asked for guidance in normalizing the data in preparation for the subsequent analysis.

- *Inspiration of ideas in the creative process*

I requested assistance in finding aesthetic references and visual ideas for the development of the web application's frontend.

- *Other uses linked to the generation of specific points of the specific development of the paper*

If you think it is necessary to include it, add it in each case:

using the *prompt* ...

(Write the request made to the AI)

having as interaction ...

(Describe what interaction you made with the AI after the prompt's response)

Example: ***I declare that*** *I have made use of the Generative AI system* ***Chat GPT 3.5*** *to* ***search for information*** *using* ***the prompt: "Tell me a concrete example that illustrates the application of the urban planning proposal of the 15-minute City in Spain"****, having as interaction* ***the proposition of the concrete example of the Chamartín district that has been reflected in the dissertation****.*

Part 3: Reflection on utility

Please provide a personal assessment (free format) of the strengths and weaknesses you have identified in the use of Generative AI tools in the development of your work. Mention if it has helped you in the learning process, or in the development or drawing conclusions from your work.

Generative AI, when used responsibly and ethically, is a very useful tool for certain stages of development in any project similar to a Bachelor's thesis. In the stages where tool selection and alternatives are necessary, it is undoubtedly a booster for the project developer's skills. The speed of research, the range of possibilities, the connection between forums where pros and cons of different options are discussed, and even the discovery of approaches one would not consider due to personal limitations—all greatly help the author to make decisions based on much broader and better-validated information across numerous sources.

The fear of impersonation or falsification of one's personal skills or aptitudes, especially in engineering fields, is in my view unfounded. All the advantages mentioned above can quickly become overwhelming if AI is misused—if one is not capable of making critical decisions or defending those decisions because they're attempting projects beyond their own professional scope.

In conclusion, I consider AI a wonderful tool to accelerate and broaden the phases of research and knowledge acquisition, much like the arrival of search engines (such as Google) once revolutionized work compared to previous formats like encyclopedias. When used arrogantly or excessively, however, it can significantly harm a project's outcome and the author's professional development, leaving one unsure of the reasons for critical decisions and not understanding the key processes.